

Report on
InterviewACE

Submitted to:

Mr. Dhruv Shah
(Assistant professor)

University Institute of Computing

Submitted by:

Pankaj Katoch (24MCI10215)

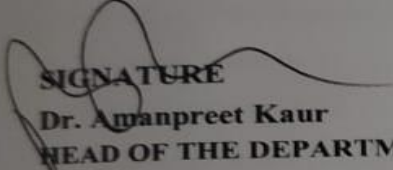
Anshul Thakur (24MCI10221)



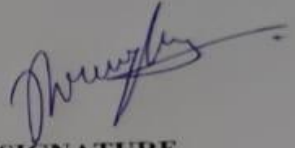
Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report "InterviewACE" is the Bonafide work of "Pankaj Katoch and Anshul Thakur" who carried out the project work under my/our supervision.


SIGNATURE

Dr. Amanpreet Kaur
HEAD OF THE DEPARTMENT
(University Institute of Computing)


SIGNATURE

Mr. Dhruv Shah
SUPERVISOR
(Assistant Professor)
(University Institute of Computing)

Submitted for the project viva-voce examination held on 16-04-2026


INTERNAL EXAMINER

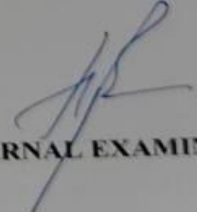

EXTERNAL EXAMINER

Table of Contents

CHAPTER 1: INTRODUCTION	1
1.1 Need Identification.....	3
1.2 Identification of Problem	4
1.3 Identification of Tasks	5
1.4 Timeline	7
1.5 Organization of the Report.....	8
CHAPTER 2: BACKGROUND STUDY	10
2.1 Timeline of the Reported Problem.....	11
2.2 Proposed Solution	12
2.3 Bibliometric Analysis	13
2.4 Review Summary	14
2.5 Problem Definition.....	14
2.6 Goals / Objectives	16
2.7 Why Use These Technologies Over Others.....	18
CHAPTER 3: DESIGN AND PROCESS.....	19
3.1 Evaluation and Selection of Specifications / Features	20
3.2 Design Constraints	22
3.3 Design Flow	24
3.4 Creation of InterviewACE	26
3.5 Logo Designing and Naming	26
3.6 Implementation Plan / Methodology	27
3.7 Database Schema Design	27
3.8 API Architecture and Endpoint Design	28
3.9 Security Architecture	29
CHAPTER 4: SNAPSHOTS	30

CHAPTER 5: RESULT ANALYSIS AND VALIDATION	34
5.1 Implementation of Solution	35
5.2 Validation.....	36
5.3 Performance Benchmarking.....	37
5.4 AI Feedback Quality Analysis	38
5.5 User Experience Evaluation.....	40
CHAPTER 6: CONCLUSION AND FUTURE WORK	42
6.1 Conclusion	44
6.2 Future Work	46
6.3 Lessons Learned.....	47
6.4 Research Contributions	48
References	50
Appendix A: Sample Question Bank.....	52
Appendix B: API Endpoint Reference.....	56
Appendix C: Prompt Engineering Templates	58
Appendix D: System Screenshots	62
Appendix E: Technical Implementation details.....	64
Appendix F: Glossary of Terms.....	66
Coursera Certification	69

ABSTRACT

When it comes to interviews people need to be good at a few things: they need to know the material they need to be able to talk about it clearly and they need to be able to stay calm. For a lot of people who are trying to become engineer technical interviews are tough because they have a time explaining what they know in a clear way even when they are not under a lot of pressure.

Most of the time the problem is not that people do not know the stuff it is just that they have a hard time talking about it in a way that makes sense. Most tools that people use to get ready for interviews do not help with this problem because they were not made to do that.

InterviewACE is different because it was made to help with this problem. It gives users questions that are related to their field they write down their answers. Then those answers are looked at by a computer program that uses the Groq inference API. The computer program looks at the answers in a different way: it checks if the answer is technically correct if it is easy to understand if it is well organized and it gives suggestions for how to make it better. Every time someone practices the whole thing is saved so they can go back and look at it later to see how they are doing.

The people who made InterviewACE chose the technology they used because it is practical and fast. The part that users interact with is made with React and Vite which makes it work quickly and smoothly without using much extra stuff. The server uses Node.js and Express.js to handle the routing and API. It stores all the information in MongoDB. They used Groq for the computer program because it is really fast which is important when people are waiting for feedback.

It took six weeks to make InterviewACE and it went through a different stage: they planned it out designed it built it put it all together tested it and then made it available to the public. This report talks about each of those stages. It explains why they made certain decisions what problems they had to solve and what they plan to do next. The version of InterviewACE that they have now is a start it shows what the idea is and it is actually helpful for people to practice and it sets the stage for what they will add next.

The people who made InterviewACE think that technical interviews are important, and they want to help people get better at them, so they made InterviewACE to help with the parts of interviews that are hardest, for people. InterviewACE is a tool that can really help people and the people who made it are excited to keep working on it to make it even better.

GRAPHICAL ABSTRACT



Figure 1: Graphical Abstract of the InterviewACE Platform

CHAPTER 1: INTRODUCTION

The software engineering hiring process has become a lot tougher over the ten years. Software engineering hiring process has become a lot tougher over the ten years. Now people who want to work in software engineering need to show that they can do a lot of things at the time. They need to be able to think through a problem loud explain their ideas clearly when they are under a little pressure answer follow-up question without getting upset and talk about their design choices in a way that someone who an expert is not can understand. This is because software engineering teams work together talk to each other and need to be able to explain why they made decisions to their colleagues.

The tools that people use to get ready for interviews were made for a purpose. Websites like LeetCode, HackerRank and GeeksForGeeks are great for helping people learn how to write code and solve problems. They are very popular. People use them a lot when they are getting ready for interviews. This is because they have a lot of problems to solve levels of difficulty and they can check your work right away and tell you if it is right or wrong. They only check if your answer is correct. They do not look at how you explain your ideas if your thinking is organized or if you can defend your design choices. These are the things that people have trouble within interviews. They are the things that most tools do not help people practice.

This means that there is a gap between how people prepare for interviews and what they actually need to do well in an interview. People spend a lot of time working on coding challenges they get good at coding. Then they get to the interview, and they realize that they need to be able to do different things. This is not something that just happens to a people. It happens to a lot of people. It is talked about in online communities where people help each other get ready for interviews. It is why some people who are very good at coding do not do well as they should in interviews. They prepared for one kind of test. They are being judged on something else. Software engineering hiring process is looking for more, than coding skills. Software engineering hiring process wants to see if you can work with a team and communicate your ideas.

InterviewACE was created to fill a need. This platform lets people practice answering interview questions in an environment. It also has a feature that uses intelligence to evaluate the answers and give feedback that is specific and detailed. The evaluation looks at the parts of the answers and how well they are explained and organized. People who use InterviewACE can see how they are doing over time and find out what they are getting better at and what they still need to work on.

When we were designing InterviewACE we wanted it to be useful above all else. We looked at every feature and every design choice. Asked ourselves if it would help someone who is getting ready for a software engineering interview. If the answer was yes, we kept it. If not, we. Put it off or got rid of it. What we ended up with is a tool that does a few things well instead of trying to do a lot of things that are just okay. InterviewACE is, about helping people prepare for software engineering interviews and that is what we focused on.

1.1 Need Identification

Establishing the need for InterviewACE is straightforward. The gap InterviewACE fills is clear in how candidates talk about their interview experiences.

They seek help. There's a big difference between the preparation tools they use and what those tools prepare them for.

A technical interview has two parts.

The first part is about knowledge: can the candidate solve the problem or explain the concept? Most preparation tools test this.

The second part is about communication: can the candidate explain their thinking in a structured way? No preparation tool helps with this.

Candidates who are strong in knowledge but weak in communication will do poorly not because they don't know the stuff. Because they haven't practiced explaining it.

Students preparing for their technical interviews are especially affected.

They may have technical foundations from school and self-study.

When they face a question that requires explaining a concept or justifying a design choice in real-time it's often their first time doing so.

Without practice the experience is tougher than it needs to be. The feedback gap makes things worse. To get better you need to know what to improve. A candidate who practices answering questions but gets no feedback, on clarity, structure or technical completeness can't improve on purpose. They may practice the weak habits over and over reinforcing them instead of fixing them. Specific consistent feedback turns practice into progress. Most preparation tools don't provide this.

The need for something to fill this gap is very real. I have seen posts, on Reddit, LinkedIn, and developer forums where people ask:

- how can I practice explaining my answers in an interview?
- how do I know if my responses make sense?
- how do I get feedback on my interview skills, not my code?

These are not minor issues. Many candidates preparing for interviews faces these problems. They want a tool that can help them prepare better.

Many careers. Universities that help students find jobs have known about this gap for a long time. They often conduct interviews to help students prepare. This shows that just knowing stuff is not enough to do well in an interview. InterviewACE makes interviews better by making them available all the time and removing the need to schedule them with someone else.

1.2 Identification of Problem

The problem is clear. People who are getting ready for software engineering interviews do not have one place where they can practice answering interview questions in a way that feels real and get helpful feedback on how well they do both in terms of getting the right answer and communicating it clearly. They also cannot see how they are getting better over time.

This is a problem with four main parts, each of which makes it harder for people to do well in interviews.

- The first part is that there is a difference between practicing coding problems on a website and being in an interview. In an interview you have to listen to a question think about what it means ask questions to make sure you understand and then give a well-organized answer. None of this happens when you practice coding problems.
- The second part is that even when people do practice answering questions, they rarely get feedback that's consistent and helpful. They do not know if their answers are good or not and they do not know how to make them better.
- The third part is that people cannot see how they are doing over time. Even if they get feedback on one practice session they cannot see if they are getting better or worse and they cannot make plans to improve.
- The fourth part is that people have to use different tools to get ready, for interviews and these tools do not work well together. This makes it hard for people to focus on getting ready.

InterviewACE was made to fix all these problems. It has a module that makes interviews feel real. It has a system that gives people feedback on how they do. It has a way for people to see how they are doing over time. And it has one interface that does everything, so people do not have to switch back and forth between different tools.

1.3 Identification of Tasks

To get the problem definition working as an application we had to break down the development work into tasks. We made a list of things that needed to be done. Then we figured out how hard each task would be. After that we decided what order to do them in. We also kept track of how we were doing on each task as we worked on the project. The tasks we did to develop InterviewACE are talked about in the sections that follow.

1.3.1 Frontend Development

The frontend of the application was built using React and Vite was the development tool that was used. We chose React because it lets us break down the interface into parts called components. This makes it easy to work on views and update one part of the interface without messing up the other parts. We picked Vite over Create React App because it starts up faster. It is better at replacing modules while we are working on the application. This makes a difference when we are working on the application for a long time.

Our main tasks for the frontend were to design and build all the views for the application. This included a landing page, a page for registering and logging in a screen for selecting a domain the main interview view, a page, for feedback and results and a dashboard for users to see their history. We used React Router to help users move between these views.

We made sure the layout would work on screen sizes from the very start. We did not wait until the end to do this. We checked to make sure everything looked good on both desktop and mobile screens as we were working on the application. This way we could catch any problems on.

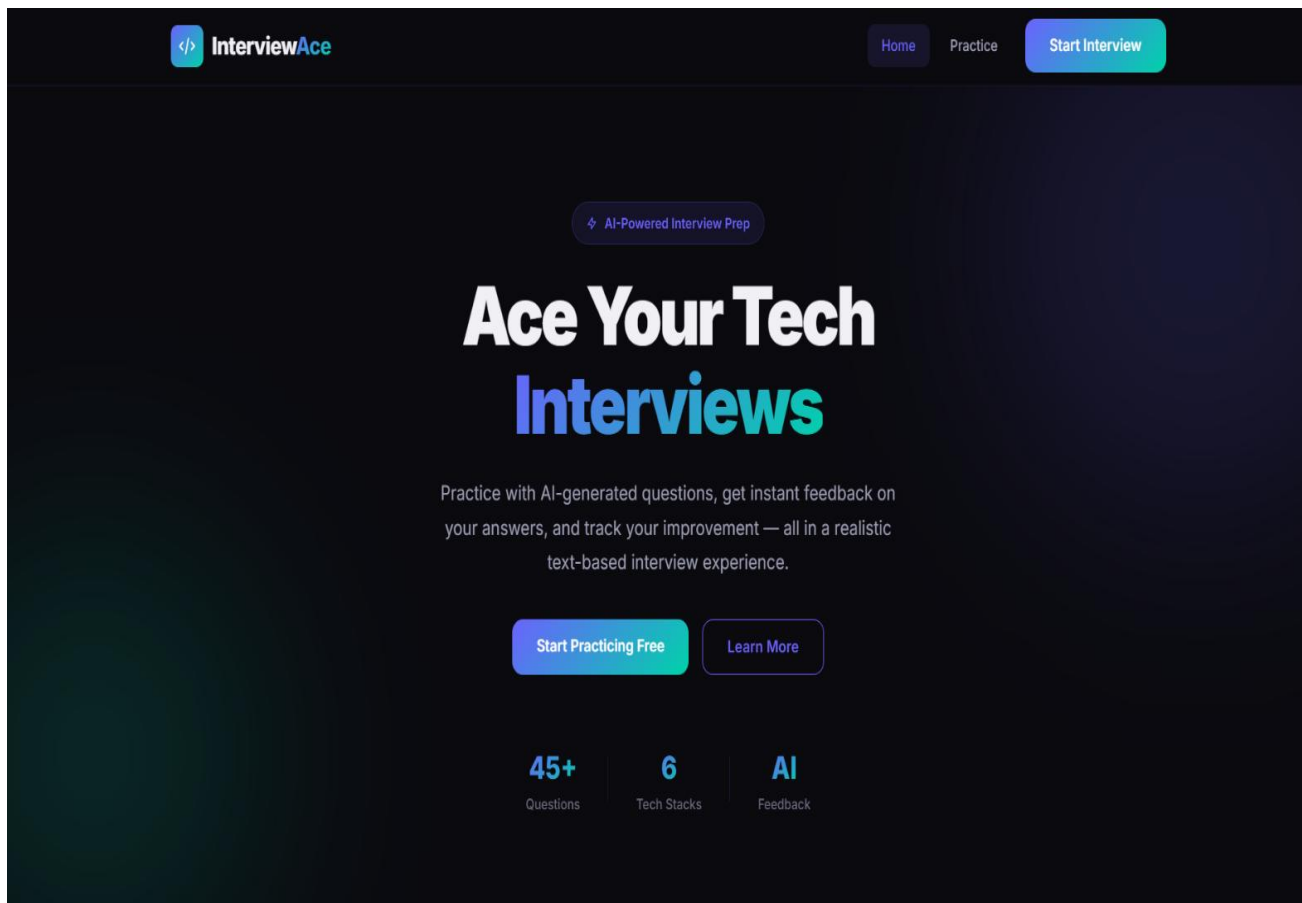


Figure 2: Frontend Interface Overview

1.3.2 Backend Development

The backend of the application was built using Node.js and Express.js was used to handle the routing and the API layer. We used JavaScript for the stack, which means both the frontend and the backend. This kept the code consistent. Made it easier to work on because we did not have to switch between two different languages.

We chose Express because it gives us the freedom to do things our way. The server side of the application had requirements and we did not need a lot of extra structure that some other frameworks would have given us.

For storing data, we used MongoDB. Mongoose helped us define the structure of the data and make sure it was correct. We had to store information, about user accounts, interview questions, session records and the answers people gave to questions.

We made REST API endpoints to support all the things the frontend needed to do. This included logging in getting questions submitting answers and looking at what happened in the past. We tested each endpoint to make sure it worked. Then we tested the whole application to make sure everything worked together. Each endpoint was tested on its own before we connected the stack and tested it all together.

1.3.3 AI Feedback Integration

The AI feedback system needed a lot of work to get it just right. It works by sending a message to the Groq inference API with the question the users answer and some instructions on how to evaluate it. The model then sends back a response with a score ratings for how accurate and clear the answer is and some tips on how to make it better.

To get this feature working we had to try a lot of versions of the message we send to the model. At first the feedback we got was either too general or too focused on details. We finally figured out a way to make the message ask for what we need. Now we get feedback that's helpful. The message asks the model to look at four things to send the response, in a special format and to make sure the tone is right. We also made sure the system can handle it if the model's response is not what we expected. This way the AI feedback system can give users feedback that's useful like the AI feedback system does. The AI feedback system is a part of the project, and the AI feedback system helps users improve their answers.

1.3.4 Deployment and Infrastructure

We used GitHub to handle version control for the project. We made a branch for each new feature or big change. This way we could work on it separately before putting it into the part of the project. Someone would review it quickly. Then we would merge it into the main part.

We put the frontend part of the project on Vercel and the backend part on Render. We used the parts of these services. Whenever we made changes and put them into the part of the project the project would automatically go live. This meant we did not have to do anything to make it happen.

We kept information like the Groq API key and the MongoDB connection string, in special environment variables. We did not put these secrets in the code. Before we first released the project we went through a checklist to make sure everything was working correctly. We checked that all the

secret information was set up right and that the whole project was working as it should from start to finish. We did this to make sure the project was ready for people to use.

1.4 Timeline

Version 1 development was structured as a six-week effort, with each week devoted to a defined area of work. The schedule was designed so that each week's output formed the input for the following week, creating a logical progression from planning through to deployment.

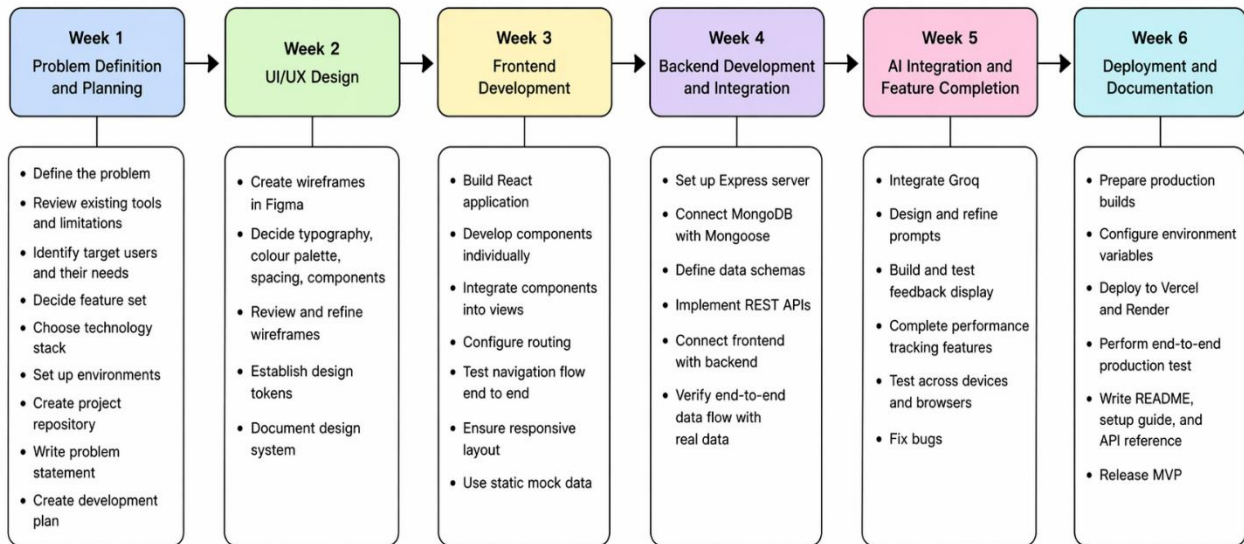


Figure 3: Six-Week Development Timeline

Week 1: Problem Definition and Planning

The first week was about figuring out the problem. We looked at the tools that're already out there and what they cannot do. We found out who the target users are and what they need. We decided on the features we want to have.

We picked the technology we will use. We set up the development environments. Made the project repository. We got the structure in place. At the end of the week, we had a written statement that says what the problem is and a plan for development that the team will follow. The team will use this development plan and the problem statement to guide the project. The problem statement and development plan are very important, to the project.

Week 2: UI/UX Design

We made wireframes for every view, in Figma. We picked the typography and the colours we would use. We also decided on the spacing and how the different parts would look. We wrote all of this down so everyone would know what to do.

When we looked at the wireframes again, we found some problems. The interface we had planned was not going to work the way we wanted it to. So, we fixed these problems before we started writing any code. We made a set of design rules that everyone would follow. This way all the views would look the same even if different people were building them. We called this set of rules a design token set. The design token set helped us make sure everything looked the same. Wireframes were a part of this process, and we used them to make sure our design token set was working.

Week 3: Frontend Development

The React application was put together from the wireframes one part, at a time. We made each part. Made sure it worked on its own before adding it to the rest of the application. We set up the way the application moves from one page to another. We tested it from start to finish. We also made sure the layout looks good on devices and screen sizes. For now, we used data to test the React application and later we will connect it to the real backend system.

Week 4: Backend Development and Integration

The Express server was set up. MongoDB was connected using Mongoose. The data schemas were also defined. The REST APIs were implemented.

By the end of the week the frontend and backend were connected. The complete data path was verified with data. This path goes from a user action in the browser through to a database write and back, to the frontend display.

Week 5: AI Integration and Feature Completion

Groq was integrated into the system. The design of the prompts was. Refined through testing. We tested the feedback display. Made sure it worked well. Features to track performance were. Tested. The entire system was tested on devices and browsers. Some bugs appeared during testing. We fixed them within a week.

Week 6: Deployment and Documentation

We got the production builds ready we set up the environment variables for the environment and we put the application on Vercel and Render.

We did a test of the production deployment to make sure that every feature of the application worked correctly when it was live.

We wrote down all the details in documents like the README, a guide to set it up and a reference, for the API. Then we released the minimum viable product or the MVP for short which is the application.

The application was thoroughly checked to ensure the production deployment was working as expected in the context and every feature of the application was working fine.

1.5 Organization of the Report

Chapter 1 is where the project is introduced. It talks about the problem the project is trying to fix. It also talks about the work that was done. How long it took to do it.

Chapter 2 is about looking at the picture. It looks at the tools that already exist to help people prepare for interviews. It also looks at what other people have researched and figured out. The problem is. The goals of the project are stated.

Chapter 3 is all about designing and developing the project. This includes picking which features to include and how the system should work. The interface is. The development process is explained.

Chapter 4 is where the results of the project are shown. It talks about how the project works and how people like using it. The project is. The results are shared.

Chapter 5 is the part of the project. It looks back at what was done. What was learned. The projects contributions to research are talked about. There is a plan for what to do next, with the project. The project is the focus, and the projects future is discussed in this chapter.

CHAPTER 2: BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The current interview preparation landscape did not emerge all at once. The gap that InterviewACE addresses developed incrementally as interviewer expectations evolved in one direction and the dominant preparation tools stayed relatively fixed in another. The timeline below traces the key developments from 2015 to 2024.

2015 to 2017: Proliferation of Online Learning Resources

The middle of the 2010s saw an increase in the amount of free stuff you could use to learn about software development. GeeksForGeeks became a popular place to go for algorithms and data structures and systems concepts. Stack Overflow was already the go to place for figuring out problems with your code. You could also find YouTube videos and online courses on websites like Coursera and edX that would teach you about things in a structured way.

The problem with this time was that all the things you could learn from were basic. You could read about programming on GeeksForGeeks or watch a video about tree traversal on YouTube, but you could not really practice using that knowledge in a situation that was, like a real job interview. The information was available. There was no way to practice what you learned. Learning software development and being able to use it were two things. You had GeeksForGeeks and Stack Overflow and YouTube and all these other resources. You still needed a way to practice to actually use software development skills.

2018 to 2019: Coding Challenge Platforms Become the Standard

LeetCode and HackerRank grew a lot during this time. Became a must for people preparing for software engineering jobs.

They had a system where they categorized coding problems by difficulty and used automated tests to check answers.

This system worked well for building coding skills. Became a big part of how companies hire tech talent.

Companies started to say that using these platforms was a requirement or at least really helpful.

But this approach had a downside. It made people think that preparing for interviews was about writing code and passing tests.

To prepare people would submit their code pass the tests and then look at the answers in the editorial. The problem with this is that it didn't teach candidates to explain their ideas or talk about their code in a way that makes sense to a person.

They got good at coding. Not so good at talking about their coding, in a job interview.

Candidates who only used these platforms got coding skills but struggled to communicate those skills when it mattered.

They couldn't explain their thinking. Structure an answer that a human would understand.

2020 to 2021: Remote Interviews Amplify the Communication Gap

The COVID-19 pandemic made remote hiring more common. This change made communication in interviews more important. In interviews candidates had to explain their thinking clearly. They had to describe solutions in words, not on a whiteboard. This was a skill for them. Many candidates weren't ready for this change. They still used automated platforms to prepare. In remote interviews they had to communicate differently. After interviews candidates talked about this on forums. They said they were technically prepared. Struggled to explain things clearly. The remote interview required more than technical skills. Candidates needed to practice explaining their thoughts. They had to get used to describing solutions in words. This was a challenge, for them. Candidates had to adapt to the interview format. They had to learn to communicate in remote interviews.

2022 to 2025: Intensified Competition Raises the Stakes

The need for software engineers is still very high but there are more people who are prepared for these jobs now. Because of this the process of interviewing for these jobs has become much harder. Now companies often have rounds of interviews including system design interviews, behavioural interviews, and technical screenings. They also have specific rules for how they conduct interviews. As a result, companies expect people who are applying for software engineering jobs to be very good at explaining their ideas.

Some people started interview groups with their friends and a few companies began offering interview coaching services to help people prepare for these tough interviews. However, these solutions do not work for everyone. The mock interview groups rely on finding someone who's willing to practice with you and who has a schedule that matches yours. The professional coaching services are expensive and only available to a people. Software engineering candidates still need a lot of help to get ready for

these interviews. The demand for software engineering talent and the need, for interview skills are still very high.

2026: The Case for an Integrated Platform

By 2026 people started to see that the way things were being done to get ready for interviews was not good enough. This was obvious to the people who were going to be interviewed and the people who were teaching them. The things that were available to help with this problem only fixed some of it not all of it. InterviewACE was thought of because there was a need for a platform that could do a thing: it could simulate real interviews give feedback from artificial intelligence on how well someone communicates and keep track of how someone does, over time. This would fill in the gaps that other tools were not able to cover.

2.2 Proposed Solution

InterviewACE is a website that puts three things together that were not available in one place before: practicing interviews using intelligence to check what people write and keeping track of how people do, over time. The way InterviewACE works is explained in detail below.

2.2.1 AI-Powered Feedback System

InterviewACE is a web tool that combines three features that weren't available in one tool before: interview practice, AI evaluation of written answers, and tracking progress over time.

The platform works like this: when a user submits an answer the system creates a prompt with the question the users answer and evaluation instructions. This prompt is sent to the Groq API, which returns a response in JSON format with a score accuracy clarity of explanation and suggestions for improvement.

The JSON format helps keep evaluations consistent and allows storing data in MongoDB for analyzing scores and feedback. The evaluation prompt was. Improved several times. Early versions gave feedback that was either too general or too technical.

The final version asks for evaluation in four areas: accuracy clarity organization and suggestions, for improvement. It also includes tone instructions that produce specific feedback. The prompt specifies field names and types to reduce parsing errors. See the subsections.

2.2.2 Interview Simulation Environment

The simulation module shows you questions one by one in a screen. This helps you give thoughtful answers. You pick a technology area. Get a set of questions for that area. You go through them one by one. For each question you write an answer in the text box before you submit it. This way of showing questions one by one is like an interview. It helps you get into the habit of giving organised and complete answers instead of just rough ideas.

We decided not to add a time limit in the version of the simulation, on purpose. When you are just starting to practice the important thing is to learn how to give clear and complete answers. If we added a time limit early, it might make you rush and not think carefully about your answers. This would make the practice less useful. We will add a timed mode in a version if you want to try it.

2.2.3 User Authentication and Data Persistence

To keep track of how people do we need to know who they are. We have to be able to save information about each person so we can look at what they did. The system that checks who people are uses email and password to sign up and codes to manage what they do when they are logged in. All the information about what people do is kept separate for each person. They have to be logged in to see it. We make passwords secret before we save them. We keep the important information about how to log in in a safe place not in the code. This way the authentication system, the system that checks who people are is safe and secure. The authentication system is very important, for the system that keeps track of what people do. The authentication system helps us keep track of what each user, each person who uses the system does.

2.2.4 Performance Tracking

When we finish a session, we save everything to MongoDB. This means we keep all the questions, what the user said, what the AI said and the scores for each question. The dashboard gets all these records. Shows them to us as a list of past sessions. We can see when it happened what it was about and how we did on average. If we want to know more we can look at any session and see what the AI said about each question. Looking at all these sessions helps us see how we are getting better, over time. It also helps us find the things we keep getting wrong so we can work on those things in the session. The MongoDB records are really important because they let us see our progress and find the

things we need to work on. We can use the MongoDB records to see how we did on each question in each session. This helps us understand what we need to do to get better.

2.2.5 Multi-Technology Domain Support

The question bank is set up by what kind of technology it's it has things like Java and Python and JavaScript and the MERN stack in the first version. When you start using it you pick the kind of technology you want to do and the backend gets the questions, for that. The way the database is set up makes it easy to add kinds of technology later when the question bank gets bigger. The part where you pick the technology on the frontend is made so it can handle a lot of options without looking messy.

2.3 Bibliometric Analysis

I did a lot of reading to understand what people are saying about InterviewACE. I wanted to see if InterviewACE is really doing something and if it is filling a gap that other people have not filled yet. I looked at papers from 2015 to 2026 on Google Scholar, IEEE Xplore, Scopus, ResearchGate and the ACM Digital Library.

When I was searching for papers, I used words like "getting ready for interviews" "computer systems that give feedback" "using artificial intelligence to test skills" "using computers to generate feedback" "testing how well people communicate in technical situations" and "evaluating online learning platforms". I picked the papers that were really relevant, to how InterviewACE was designed and built than trying to read everything. I was focused on InterviewACE and how it works so I chose papers that talked about things.

2.3.1 Volume and Trajectory of Research Output

Publications about AI, automated assessment and online learning increased a lot after 2020. The pandemic made people move to learning and remote hiring. This made more people interested in AI tools that help with evaluations and digital interviews. The number of publications grew by 40 to 50 percent from 2019 to 2023. The biggest growth was in AI evaluation and automated feedback. Published work was about checking code and multiple-choice questions. This is because it's easy to automate these tasks. There was work on evaluating written responses using AI. This area was growing fast. This growth was because large language models got better and were available through APIs. AI and automated assessment were areas of research. AI evaluation and automated feedback were

subcategories. Online learning and digital interview practice tools were also important. The growth of AI evaluation and automated feedback was very high. The use of AI, in learning was increasing. Automated assessment was becoming more popular. The quality of AI evaluation was improving. The availability of AI tools was increasing.

2.3.2 Relevant Research Contributions

Researchers at Stanford and Coursera did some work on feedback in online learning. They found out that getting feedback that is targeted to what you did is really helpful in learning new skills fast. Many studies have shown that when you get feedback that's specific to what you did and you get it right after you try something you do a lot better than if you get feedback later or if it is not specific to what you did. The InterviewACE feedback system is based on this idea.

Some other researchers did work on computer models called transformer-based language models like BERT and GPT. This work showed that it is possible to use computers to evaluate what people write in a way that's smart and understands the context. These models can look at what you write and figure out if it is correct, clear, and well organized. This makes it possible for InterviewACE to give you feedback on what you write.

There have been studies on using computers to help teach people new things. These studies have shown that when people get feedback from computers on what they do they get better at it faster. When students got feedback from computers on their work, they did better the time they tried than students who did not get this feedback. This is what InterviewACE is based on: the idea that getting feedback from computers on how you do in interviews will help you do better in real interviews. InterviewACE uses this idea to help people improve their interview skills. The researchers, at InterviewACE think that if you practice answering interview questions and get feedback from computers you will do better in interviews.

2.3.3 Identified Gaps in the Literature

The review did not find any system or research prototype that offers three features. These features are interview simulation with written answers AI evaluation of both technical and communication skills and tracking individual performance over time. There are systems that offer one or two of these

features. However, none offer all three together. This gap shows that InterviewACE is a useful tool.

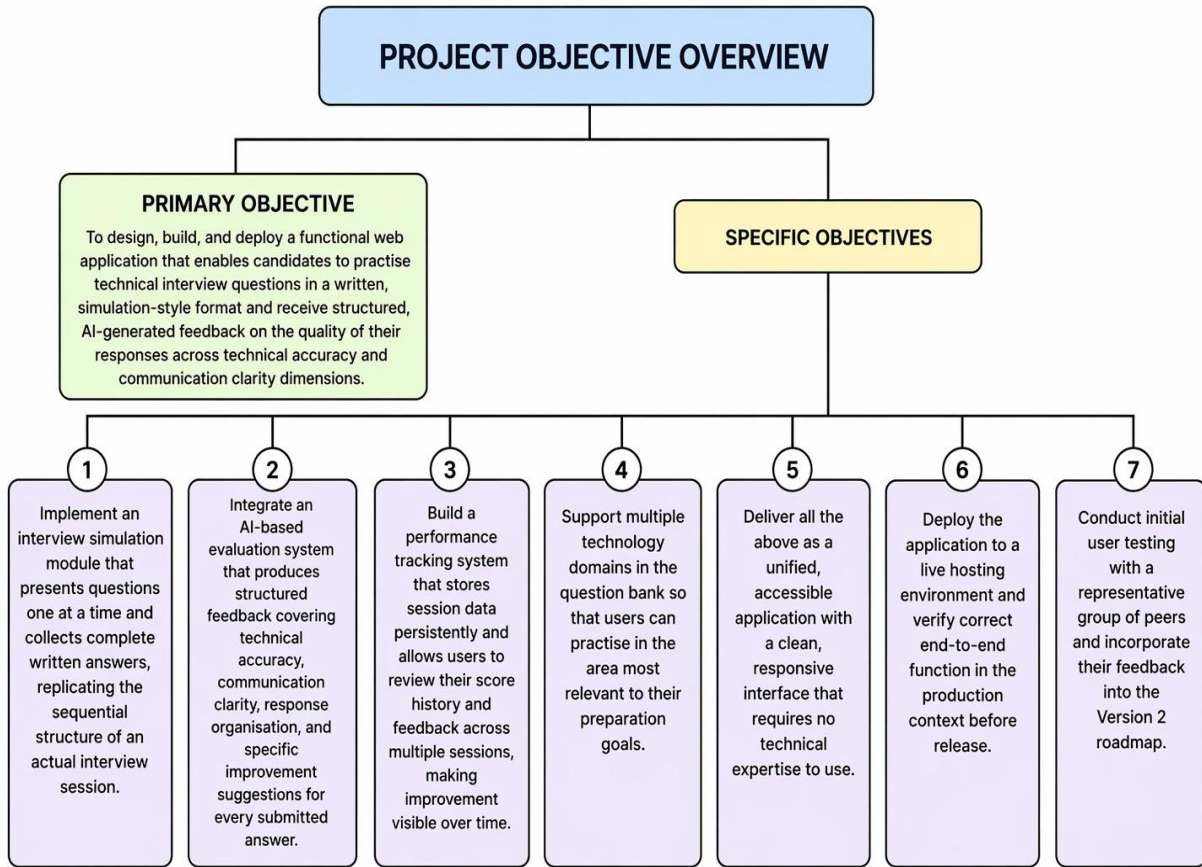


Figure 4: Project Objectives Overview

2.4 Review Summary

The review of the tools and research confirms two main points. First the problem that InterviewACE solves is real and documented. Current tools used by candidates leave gaps in preparation. These gaps affect how well candidates do in interviews.

Second the approach used in InterviewACE. Using AI to evaluate written responses and give feedback. Is solid. It has shown results in similar educational settings. There is no existing tool or research that offers the combination of features as InterviewACE. It fills a gap in the preparation landscape. Many people preparing for interviews are frustrated and look for workarounds because there is no tool. The demand, for InterviewACE is consistent and substantial.

2.5 Problem Definition

Having reviewed the situation and existing resources the problem is clear. People getting ready for software engineering interviews do not have one place that offers realistic written interview practice. They also do not get feedback on both technical accuracy and how well they communicate for each response they submit. Furthermore, they cannot track their progress over time across practice sessions. The effects of this gap are significant. They cause candidates to perform worse in interviews than they can. This happens because their preparation does not focus on the skills that interviews really test. It also means that practice does not lead to improvement. This is because the feedback needed to guide improvement is not available. Additionally, candidates cannot find out. Fix their specific weaknesses. This is since there is no record of what those weaknesses are.

These issues are not due to candidates not trying hard enough or not being good enough. They are the result of a gap in the tools available, for preparation. A platform that fills this gap will help candidates do better in interviews. That is what InterviewACE is designed to do.

2.6 Goals / Objectives

2.6.1 Primary Objective

We need to create a web application that helps people who are looking for a job. This web application will allow them to practise answering interview questions. They will do this by writing their answers. Then the web application will give them feedback. The feedback will be about how good their answers are from a technical point of view and how well they are able to communicate their ideas. The web application will use intelligence to give this feedback. The goal is to make sure people can get better at answering interview questions by using this web application. We want to make it easy for them to use and get feedback. The web application will be available, on the internet. People can use it to prepare for their job interviews.

2.6.2 Specific Objectives

1. Implement an interview simulation module that presents questions one at a time and collects complete written answers, replicating the sequential structure of an actual interview session.

2. Integrate an AI-based evaluation system that produces structured feedback covering technical accuracy, communication clarity, response organisation, and specific improvement suggestions for every submitted answer.
3. Build a performance tracking system that stores session data persistently and allows users to review their score history and feedback across multiple sessions, making improvement visible over time.
4. Support multiple technology domains in the question bank so that users can practise in the area most relevant to their preparation goals.
5. Deliver all the above as a unified, accessible application with a clean, responsive interface that requires no technical expertise to use.
6. Deploy the application to a live hosting environment and verify correct end-to-end function in the production context before release.
7. Conduct initial user testing with a representative group of peers and incorporate their feedback into the Version 2 roadmap.

2.7 Why Use These Technologies Over Others

2.7.1 React Over Angular or Vue

React was chosen because it has a component model. The team already knew how to use it. It also has libraries and community resources. Angular was. Not chosen. It requires a lot of setup and structure. This would have slowed us down at the start of the project. Vue.js could have worked too. React has a bigger developer community and more documentation. This helps a lot when dealing with problems, during development. We used Vite of Create React App. Vite starts faster. Has better hot module replacement.

2.7.2 Node.js and Express Over Firebase or Django

Using Node.js on the backend lets us use JavaScript which makes it easier to work on both frontend and backend code. This way we do not have to switch between programming languages and can share data structures easily. We picked Express over Node.js frameworks because we knew exactly what we needed from the server-side. The requirements were clear, so we did not need rules, from a framework. We also thought about using Firebase as a managed backend option. We decided against it because

we needed to customize how we built and handled AI prompts and responses. For this we needed a server that we could control fully which Firebase did not offer.

2.7.3 MongoDB Over PostgreSQL

The MongoDB document model is really good for this project because it has a lot of nested data. The interview session records have a lot of question and answer and feedback things in them. These things can be easily stored in documents in MongoDB. We do not have to do a lot of work to connect them. We also thought about how the data model might change over time. This project changed a lot during development and MongoDB was very good at handling these changes. We did think about using PostgreSQL. It would have been good if we had complicated data that was related to each other. For this project MongoDB was better because it is fast and flexible. MongoDB helped us finish the project quickly. The flexibility of MongoDB was an advantage, for us. We are happy that we chose MongoDB for this project.

2.7.4 Groq Over OpenAI or Anthropic APIs

Groq was selected for a specific practical reason: its inference latency is markedly lower than the alternatives tested. Feedback generation is a real-time feature—users submit an answer and wait for the evaluation before proceeding. In testing, Groq returned responses consistently faster, and the difference was perceptible in the user experience. For a feature where responsiveness is central to how the platform feels to use, this latency advantage outweighed other considerations.

2.7.5 Vercel and Render Over Self-Hosted Infrastructure

Vercel and Render were selected because they provide reliable free-tier hosting with continuous deployment from GitHub. Both platforms handle HTTPS, deployment rollback, and environment variable management automatically. For a student project at this scale, managing server infrastructure would have been a significant distraction from the application development work itself. These platforms remove that concern entirely and produce a professional deployment without any infrastructure overhead.

CHAPTER 3: DESIGN AND PROCESS

3.1 Evaluation and Selection of Specifications / Features

Feature selection was driven by the problem definition. Every proposed feature was evaluated against the same question: does this directly address one of the four gaps—simulation, feedback, tracking, integration—identified in the problem definition? Features that did not map clearly to one of these gaps were deferred. Features that were necessary dependencies for gap-addressing features were included. Features that were desirable but would overrun the six-week timeline were documented for Version 2.

3.1.1 Authentication and User Management

Authentication was a non-negotiable prerequisite for performance tracking and was included from the beginning. The implementation was kept deliberately simple for Version 1: email and password registration with token-based session management. More sophisticated mechanisms—OAuth social login, JWT with refresh token rotation, two-factor authentication—were identified as Version 2 improvements and deferred.

3.1.2 Interview Simulation Module

The simulation module was the feature most directly connected to the primary problem and received the highest development priority. The Version 1 implementation presents questions sequentially, collects full written answers, and triggers AI evaluation on submission. A timer mode was scoped but excluded from Version 1 on the basis that time pressure during early-stage practice is more likely to discourage thoughtful engagement than to improve it. The timer will be an optional feature in Version 2.

3.1.3 AI Feedback System

Developing the AI feedback system required the most sustained design effort of any feature in the project. The prompt design went through at least six iterations before the final version was stable. Early versions produced feedback that was either too high-level to be actionable—comments like "improve your explanation" without specifying how—or too narrowly focused on technical correctness while ignoring the communication dimension. The final prompt structure specifies four

named evaluation dimensions and requests a JSON response with exactly defined field names, which produces consistently specific, parseable, and useful feedback.

3.1.4 Question Management System

Questions are stored in MongoDB with fields for question text, technology category, difficulty level, and topic tags. For Version 1, a manually curated seed set of fifteen to twenty questions per category was created. The database structure accommodates additional questions and categories with no schema changes required. A simple admin interface for managing questions is planned for Version 2.

3.1.5 Performance Tracking Dashboard

The dashboard was implemented in a functional but intentionally minimal form for Version 1. It shows the user's session history with date, category, and average score per session, and allows drill-down to the full feedback for any session. Graphical score-trend charts were deferred to Version 2 based on the timeline constraint. The data model was designed from the start to support these charts when they are added.

3.2 Design Constraints

3.2.1 Technical Constraints

The dependency on an external AI inference API is the most significant technical constraint in the system. The quality, latency, and availability of feedback depend on Groq's infrastructure. The free tier of Groq's API imposes rate limits on requests per minute, which is sufficient for development and initial testing but would require a paid tier for sustained production use at scale. Robust error handling was implemented throughout the feedback flow to manage slow responses, rate limit errors, and malformed response formats gracefully.

3.2.2 Platform Compatibility

The application needed to function correctly on both desktop and mobile browsers. Mobile required particular attention on the interview interface, where the virtual keyboard can push the submit button below the visible viewport. This was resolved with a sticky footer layout for the primary action buttons, which keeps them visible regardless of keyboard state. Cross-browser testing was conducted on Chrome, Firefox, Safari, and Edge to ensure consistent rendering.

3.2.3 Budget Constraints

The project operated without a budget for external services. All third-party platforms were accessed through free tiers: Groq for AI inference, MongoDB Atlas for the database, Vercel for frontend hosting, and Render for backend hosting. All core features were implemented successfully within these free-tier constraints. The rate limits of the free tiers would require upgrade for a production deployment serving a significant number of concurrent users, but they are fully adequate for demonstration and testing purposes.

3.2.4 Privacy and Security

User passwords are hashed with bcrypt before storage. API keys and database credentials are stored as environment variables and are not present in the codebase or version history. User session data is access-controlled and cannot be retrieved without a valid authenticated session. The only user data shared with a third party is the question text and the user's answer, which are sent to Groq for evaluation. No personally identifiable information is included in these requests.

3.3 Design Flow

3.3.1 User Interface Flow

The user experience follows a clear linear path. A new user arrives at the landing page, registers an account, and logs in. After login, they are taken to the dashboard, which shows their session history. From the dashboard they can begin a new session, which takes them to the domain selection screen. After selecting a domain they enter the interview interface, where questions appear one at a time. The user writes and submits each answer, receives the AI feedback immediately, and can then proceed to the next question. After the final question, a session summary is displayed. The user can then return to the dashboard or begin another session.

3.3.2 System Data Flow

When a user submits an answer, the following sequence executes. The browser sends a POST request to the answer submission endpoint, including the session ID, question ID, and answer text. The backend validates the request and retrieves the question text from MongoDB. It then constructs the evaluation prompt and sends it to the Groq API. The API returns a structured JSON response. The

backend parses the response, validates the structure, stores the complete record in MongoDB, and returns the parsed feedback to the browser. The browser updates the display with the feedback content.

3.3.3 UX Design Principles

Three principles guided every interface decision. Focus means keeping the interview screen free of distractions so that users can concentrate entirely on constructing their answers. Clarity means making the user's next action and the information being presented to them unambiguous at every point in the flow. Consistency means applying the same visual language and interaction patterns across all views so that users never have to learn new conventions as they navigate.

3.4 Creation of InterviewACE

"Mysti" is the internal designation for the AI feedback persona embedded in InterviewACE's evaluation system. It refers not to a separate model but to the specific prompt configuration that shapes how the underlying language model reads and evaluates an interview answer. The persona name was chosen to reflect the quality that good interview feedback should have: it surfaces things about a candidate's performance that they themselves may not have been aware of.

Developing this configuration required the most sustained iteration of any technical work in the project. The key insight that emerged from testing was that specifying the output structure as a JSON object with explicitly named fields—rather than requesting free-form narrative feedback—produced dramatically more consistent results. This structural specification also enabled each aspect of the feedback to be stored as a discrete database field and aggregated for analytics.

The tone of the feedback was explicitly specified in the prompt as constructive and encouraging. The model was instructed to frame identified weaknesses as specific opportunities for improvement rather than as deficiencies, and to maintain a supportive tone throughout. This was a deliberate design choice based on the understanding that discouraging feedback reduces engagement with practice, while constructive feedback encourages it.

3.5 Logo Designing and Naming

The name InterviewACE communicates the platform's purpose and aspiration in two words. "Interview" grounds it in context; "ACE" implies mastery and confident performance rather than mere adequacy. Several alternatives were considered: Interviewer was too generic, PrepAce was too

abstract, MockIQ sounded clinical and unappealing. InterviewACE was preferred as the most direct and most motivating of the options considered.

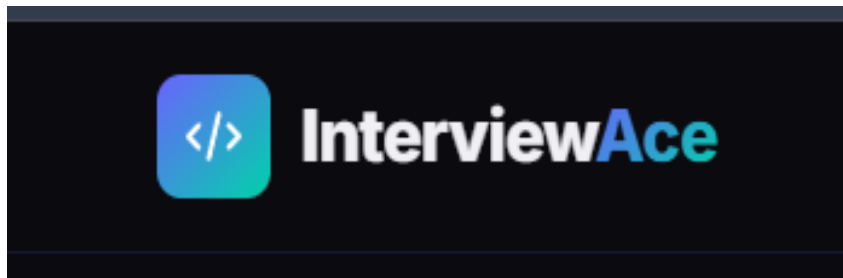


Figure 4: InterviewACE Logo

The logo was designed to reflect a professional, focused identity. Clean typography, a restrained visual style, and no decorative elements that might date the design or limit its application across contexts. It was created in Figma and refined through several iterations. The final design works consistently from favicon size up to large header dimensions, uses the platform name as the primary visual element, and communicates seriousness and purpose without being austere.

3.6 Implementation Plan / Methodology

Development followed an agile approach with weekly sprint cycles. Weekly planning set goals for the coming sprint; end-of-week reviews assessed progress and adjusted priorities for the following sprint. This structure allowed the team to respond to what they learned during development—particularly during the AI integration phase, where prompt design required more iteration than initially estimated—without abandoning the overall plan.

Phase 1: Planning and Requirements

The planning phase produced a precise written problem statement, a target user profile, a prioritised feature list with clear in-scope and out-of-scope distinctions, and a finalised technology stack. These were documented and reviewed before any code was written, establishing a shared understanding of the project scope.

Phase 2: UI/UX Design

Wireframes were built in Figma for every major view. Design reviews—conducted as structured walkthroughs of the wireframes—identified usability issues before implementation began. A design

token set covering colour, typography, and spacing was established to maintain visual consistency as the interface was built by two developers.

Phase 3: Frontend Development

React components were built from the wireframes, starting with the simplest views and progressing to the more complex ones. Each component was tested in isolation before integration. Routing, navigation, and responsive layout were verified at the end of the phase using real devices in addition to browser developer tools.

Phase 4: Backend Development

The Express server, MongoDB connection, data schemas, and REST APIs were implemented in sequence. Integration testing connected the frontend to the backend at the end of the phase, verifying the complete data path with real data.

Phase 5: AI Integration

Groq was integrated and prompt iteration proceeded in parallel with frontend feedback display development. The final prompt configuration was fixed after six iterations. Parsing and storage logic was implemented and validated against multiple question and answer types, including edge cases.

Phase 6: Testing and Bug Fixing

Functional, cross-device, and cross-browser testing was completed. All identified issues were fixed within this phase. The most significant fix was a sticky footer on mobile to keep action buttons visible when the keyboard is open.

Phase 7: Deployment

Production builds were prepared, environment variables were configured for Vercel and Render, and the application was deployed. End-to-end testing in the production environment confirmed correct operation.

Phase 8: Initial User Feedback

The application was shared with five peer testers for structured feedback sessions. Their responses confirmed the core value of the feedback feature and identified the question bank size and graphical analytics as the top priorities for Version 2.

3.7 Database Schema Design

A carefully considered database schema is the backbone of any application that needs to store, retrieve, and analyse structured data. For InterviewACE, four primary collections were designed in MongoDB: Users, Questions, Sessions, and Feedback. The schema design had to balance immediate functional requirements with the extensibility needed to support planned Version 2 features without requiring disruptive migrations.

3.7.1 User Collection

The User document captures the minimal account information required for authentication and personalisation. Each user document stores a unique identifier, a display name, an email address used for login, a bcrypt-hashed password, an account creation timestamp, and a last-login timestamp. No redundant profile information is stored at the account level in Version 1. Role-based access control fields are defined in the schema but not currently populated, in preparation for the administrative user feature planned for Version 2.

Field	Type	Description
_id	ObjectId	Unique MongoDB document identifier
name	String	User display name
email	String	Unique login email address
passwordHash	String	bcrypt hash of user password
createdAt	Date	Account registration timestamp
lastLogin	Date	Most recent login timestamp
role	String	User role: "user" or "admin"

Table 1: User Collection Schema

3.7.2 Question Collection

Questions are the core content of the platform. Each question document stores the question text, a technology domain tag (e.g., "JavaScript", "Python", "Java", "MERN"), a difficulty classification (beginner, intermediate, advanced), one or more topic tags for granular filtering, and a boolean flag indicating whether the question is currently active in the rotation. The topic tags allow the

recommendation engine planned for Version 2 to surface questions in areas where a user has historically scored lower, creating a personalised practice experience.

Field	Type	Description
_id	ObjectId	Unique document identifier
questionText	String	Full interview question text
domain	String	Technology domain tag
difficulty	String	beginner / intermediate / advanced
topics	[String]	Array of topic tags
isActive	Boolean	Whether question is in active rotation
createdAt	Date	Date question was added

Table 2: Question Collection Schema

3.7.3 Session Collection

A Session document captures the context and summary of a single practice session. It references the user who completed the session, records the selected domain and start and end timestamps, stores the list of question IDs presented during the session, and contains a computed average score across all evaluated answers. The session document is created when the user starts a session and updated as answers are submitted and evaluated.

3.7.4 Feedback Collection

The Feedback document is created for each question-answer pair within a session. It stores the question ID, the session ID, the user's answer text, the score awarded by the AI evaluation, and the four structured feedback fields: technical accuracy assessment, communication clarity comment, structural organisation note, and improvement suggestions. Each field is stored independently rather than as a single text block, which enables field-level analytics and is what makes the per-dimension score trend analysis planned for Version 2 technically feasible.

3.7.5 Indexing Strategy

MongoDB indexes were defined on the fields most frequently used in query filters. The email field on the User collection is indexed with a unique constraint to enforce uniqueness and accelerate login

lookups. The `userId` and `createdAt` fields on the `Session` collection are indexed to support the dashboard query that retrieves a user's session history in reverse chronological order. The `domain` and `difficulty` fields on the `Question` collection are indexed to support the question retrieval queries that run at the start of each session. These indexes ensure that all frequently executed queries operate in logarithmic time rather than linear scan time as the collections grow.

3.8 API Architecture and Endpoint Design

InterviewACE's backend exposes a RESTful API to the frontend application. The API was designed following REST conventions: resources are identified by nouns in the URL path, HTTP methods express the operation being performed, and response codes communicate the outcome. All endpoints return JSON. Authentication is enforced on all protected routes through a middleware layer that validates the session token before allowing the request to proceed.

3.8.1 Authentication Endpoints

Method	Path	Description
POST	/api/auth/register	Create a new user account with email, name, and password
POST	/api/auth/login	Authenticate with email and password, return session token
POST	/api/auth/logout	Invalidate the current session token
GET	/api/auth/me	Return the authenticated user's profile data

Table 3: Authentication API Endpoints

3.8.2 Question Endpoints

Method	Path	Description
GET	/api/questions/:domain	Retrieve question set for specified domain
GET	/api/questions/domains	List all available technology domains

Table 4: Question API Endpoints

3.8.3 Session and Feedback Endpoints

Method	Path	Description
POST	/api/sessions/start	Create a new session record for the authenticated user
POST	/api/sessions/:id/answer	Submit an answer and receive AI feedback
POST	/api/sessions/:id/complete	Mark session as complete, compute average score
GET	/api/sessions	Retrieve session history for authenticated user
GET	/api/sessions/:id	Retrieve full detail for a specific session

Table 5: Session and Feedback API Endpoints

3.8.4 Middleware Architecture

The Express application is structured with a layered middleware stack. The outermost layer handles CORS configuration, JSON body parsing, and request logging. The authentication middleware layer validates session tokens and attaches the authenticated user object to the request context for use by route handlers. Route-specific middleware performs input validation and sanitisation before the request reaches the handler logic. Error handling middleware at the bottom of the stack catches unhandled errors and returns consistent error response structures.

This layered approach keeps each middleware component focused on a single concern, makes the authentication enforcement straightforward to apply consistently, and ensures that input validation happens at the boundary of the application before any business logic executes.

3.9 Security Architecture

Security was considered as a first-class concern throughout the design and implementation, not an afterthought added before deployment. The following describes the security measures built into the Version 1 system and the planned improvements for Version 2.

3.9.1 Password Security

User passwords are never stored in plain text. When a user registers, the submitted password is hashed using bcrypt with a work factor of 12 before being written to the database. The work factor determines the computational cost of the hash operation, making brute-force attacks against the stored hashes

computationally expensive. When a user logs in, the submitted password is compared to the stored hash using bcrypt's comparison function, which is resistant to timing attacks. The plain text password is never retained in memory beyond the duration of the request handler.

3.9.2 Session Token Management

Session tokens are generated as cryptographically random strings using Node.js's built-in crypto module. Tokens are transmitted via HTTP headers rather than URL parameters to prevent them appearing in server logs. The authentication middleware validates the token on every protected request. Token expiry is enforced to limit the window of exposure if a token is compromised. In Version 2, this will be upgraded to a proper JWT implementation with refresh token rotation.

3.9.3 Input Validation and Sanitisation

All inputs received by the API are validated and sanitised before processing. Required fields are checked for presence and type. String fields have maximum length constraints to prevent unreasonably large payloads. The answer text field is sanitised to prevent injection of malicious content before it is included in the prompt sent to the Groq API. Mongoose schema validation provides a final layer of protection at the database write level, rejecting documents that do not conform to the defined schema.

3.9.4 Dependency Management

All npm dependencies are pinned to specific versions to prevent unexpected breaking changes from upstream updates. The dependency list is reviewed before each release for known security vulnerabilities using npm audit. Only packages with active maintenance and no critical vulnerabilities are included. The number of third-party dependencies is kept to the minimum required for the application's needs, reducing the attack surface.

3.9.5 Environment Variable Discipline

Every sensitive configuration value—the Groq API key, MongoDB connection string, JWT secret, and session signing key—is stored as an environment variable and loaded at runtime. None of these values appear in the source code, the .env files are listed in gitignore and have never been committed, and the production environment variables are configured through the respective hosting platform dashboards rather than through files in the deployment package.

CHAPTER 4: SNAPSHOTS

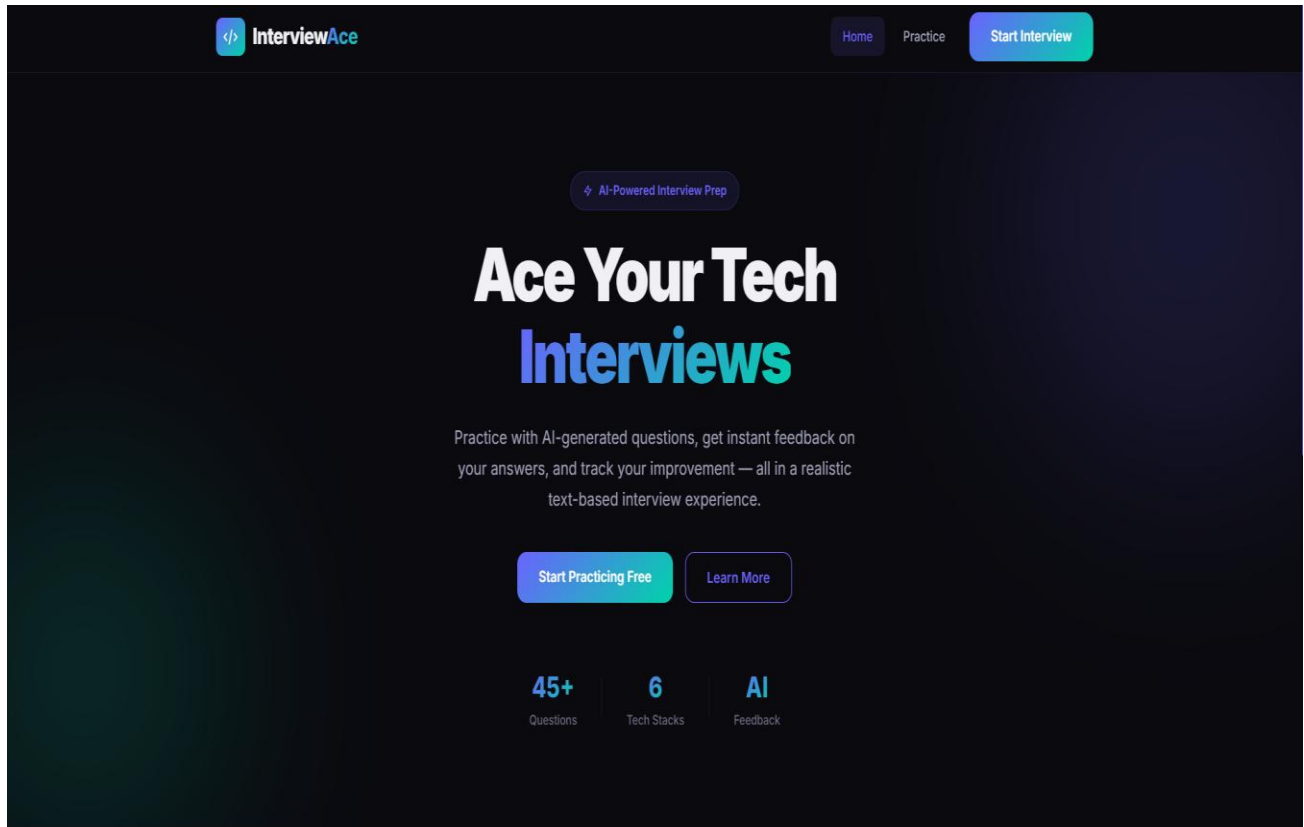


Figure 5 Home Page

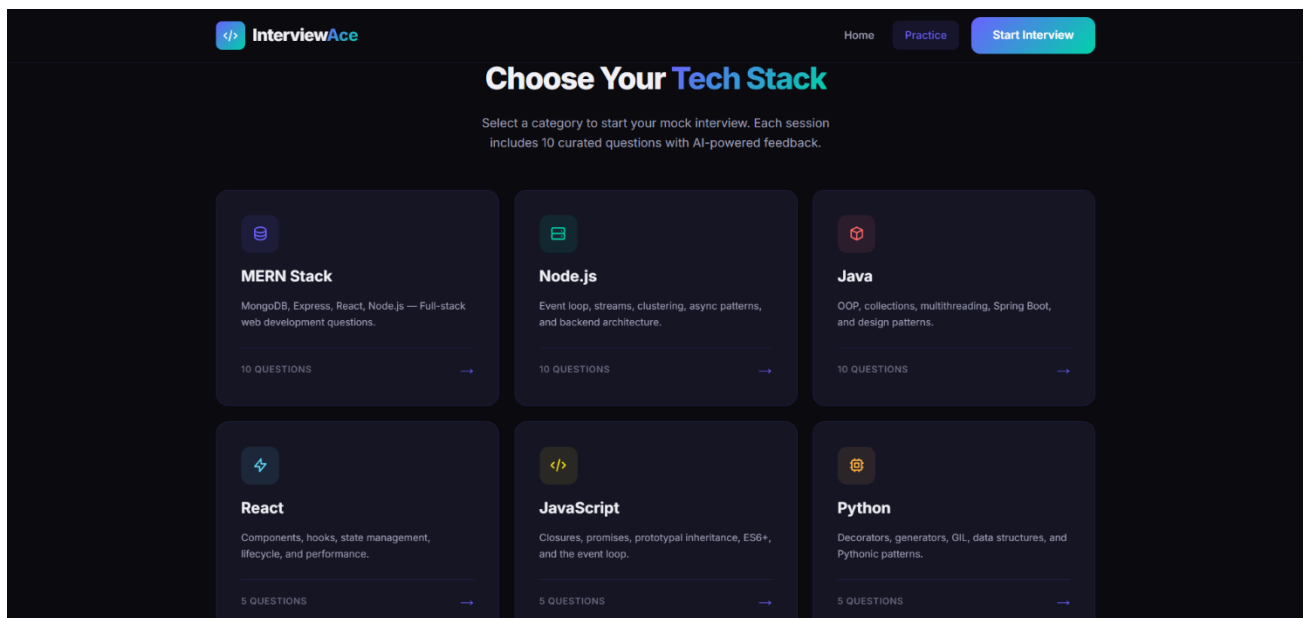


Figure 6 Tech Stack

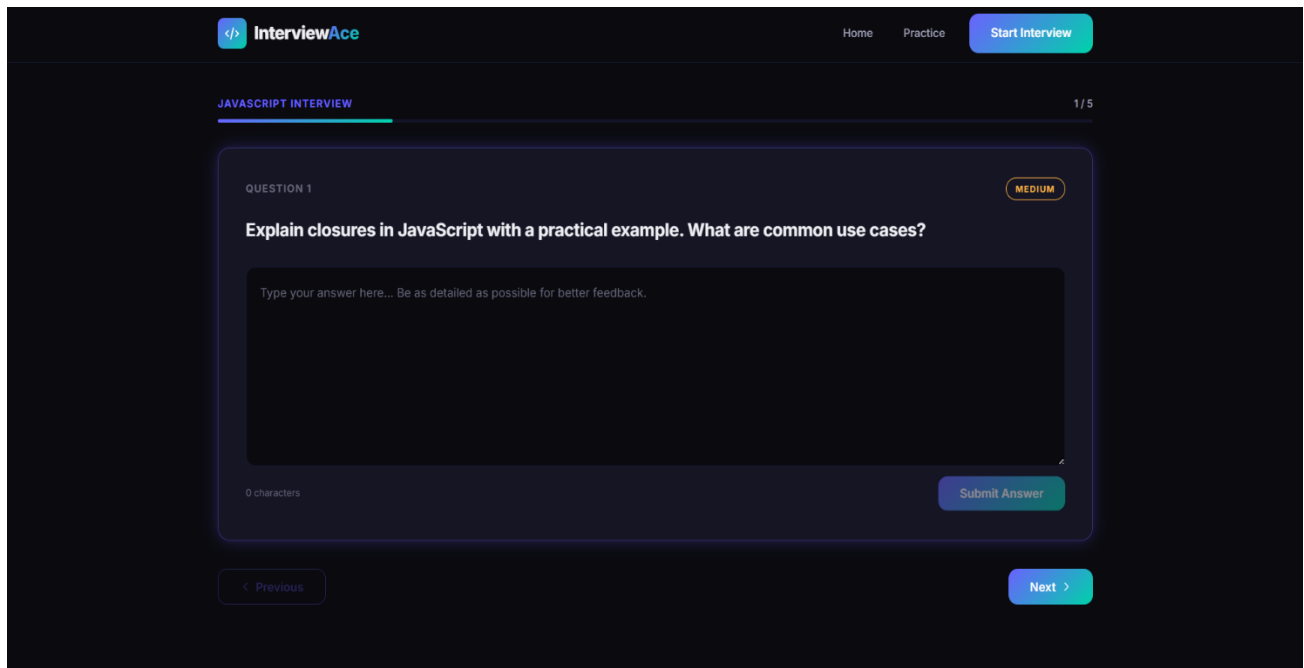


Figure 7 Question

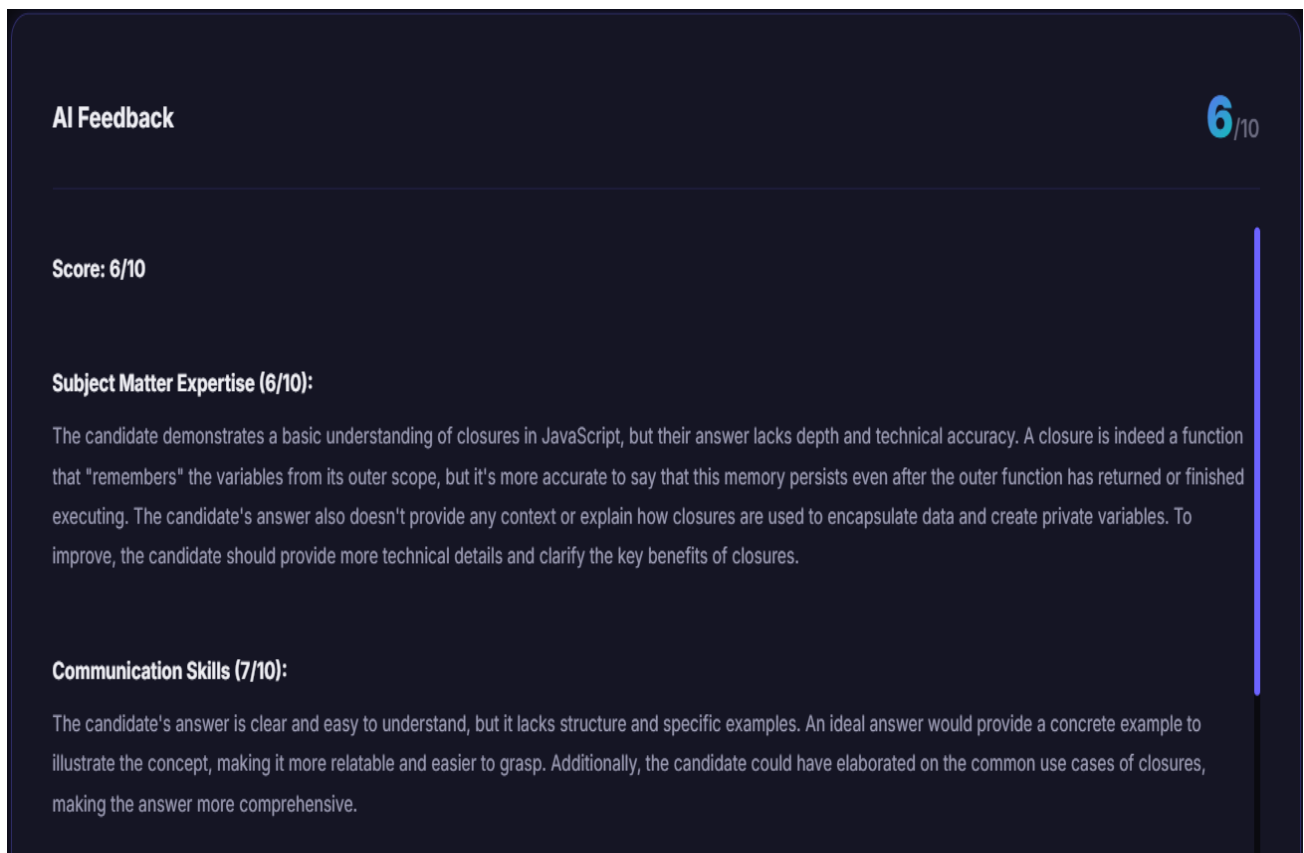


Figure 8 Feedback 1

Strengths:

1. Basic understanding: The candidate demonstrates a basic understanding of closures and their concept.

2. Clear articulation: The answer is clear and easy to understand.

Areas for Improvement:

1. Depth and technical accuracy: The candidate's answer lacks technical details and accurate explanations, making it seem superficial.

2. Lack of examples: The answer could benefit from a concrete example to illustrate the concept of closures.

3. Incomplete discussion: The candidate doesn't elaborate on common use cases, making the answer feel incomplete.

Ideal Answer Summary:

An ideal answer would provide a clear, concise definition of closures in JavaScript, accompanied by a practical example to illustrate the concept. The candidate would also discuss the benefits of closures, such as encapsulating data and creating private variables, and provide several common use cases in JavaScript development, like event handlers and module patterns.

Figure 9 Feedback 2

</> InterviewAce

HomePracticeStart Interview

✓

Interview Complete!

JAVASCRIPT Interview

7

/10

Good

You answered 4 questions in this session.

Detailed Breakdown

Q1

Explain closures in JavaScript with a practical example. What are common use cases?

6/10

Q2

What is the difference between var, let, and const? Explain hoisting and temporal dead zone.

8/10

Figure 10 Feedback 3

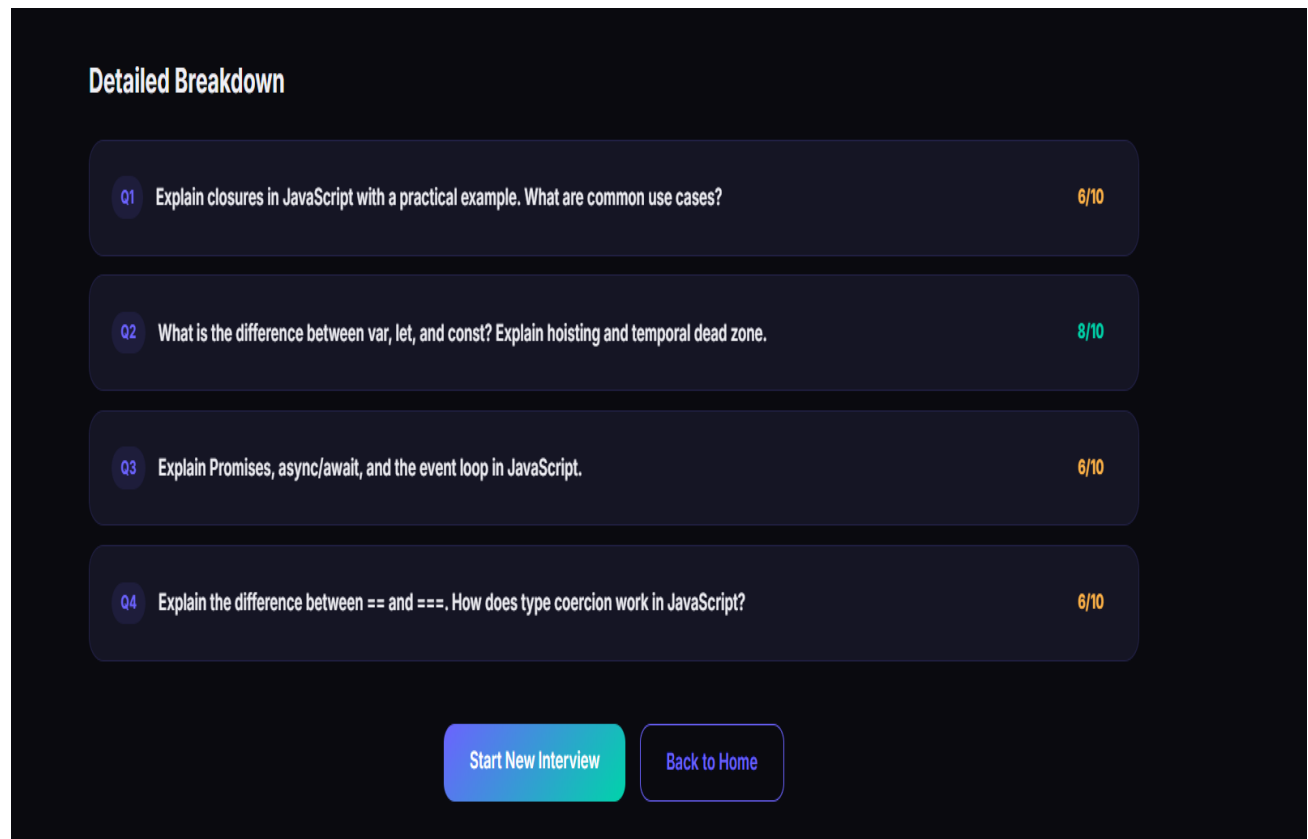


Figure 11 Feedback 4

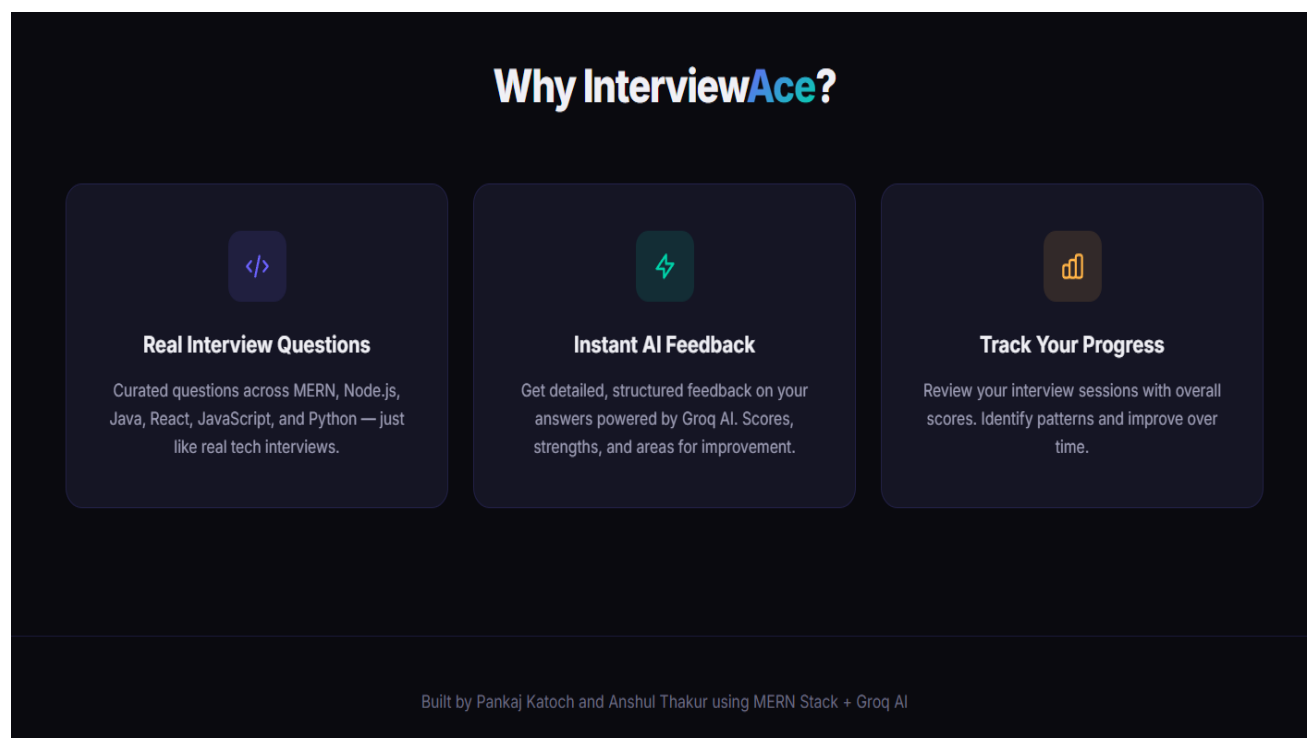


Figure 12 Benefits

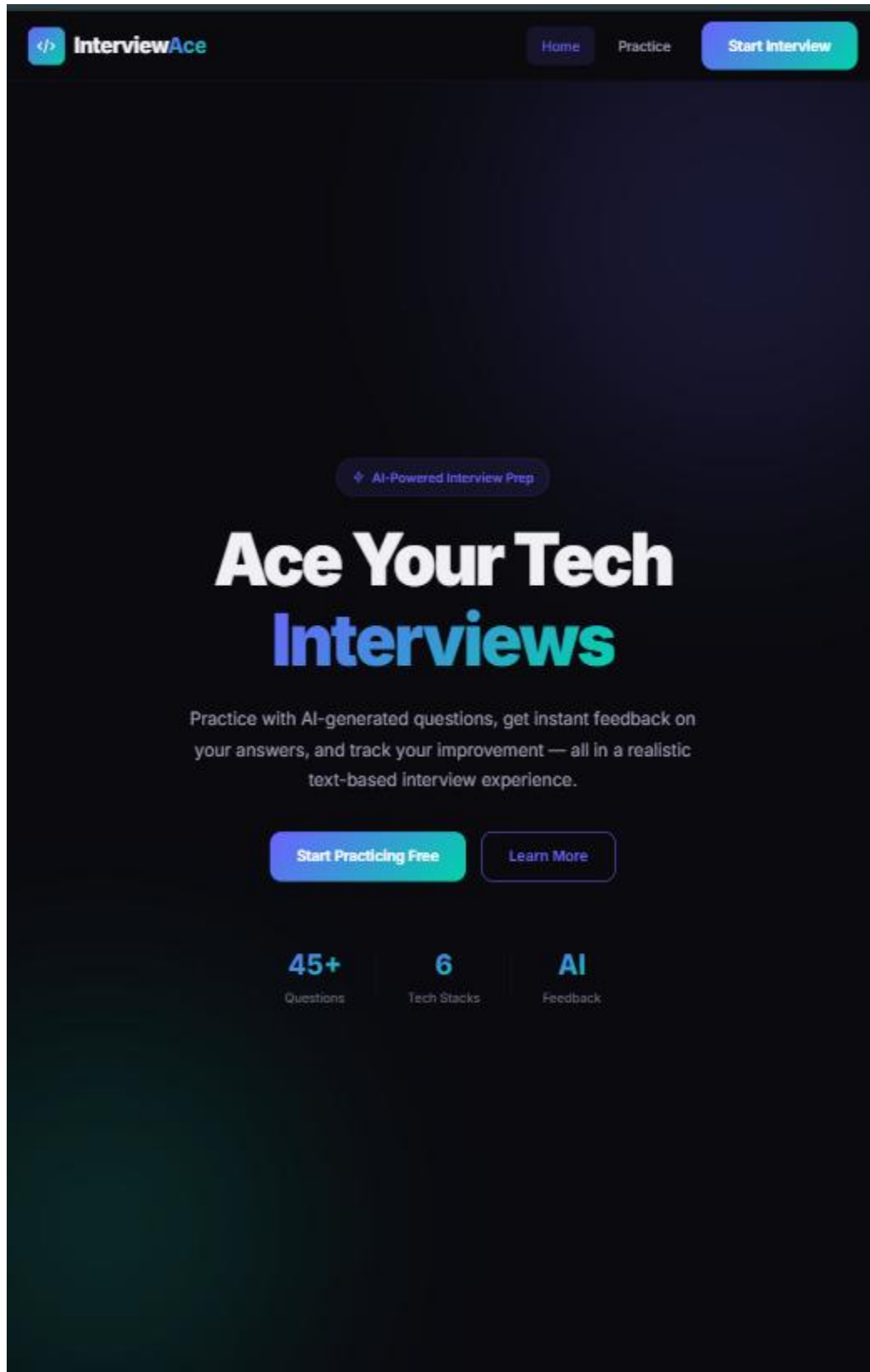


Figure 13 Mobile View

CHAPTER 5: RESULT ANALYSIS AND VALIDATION

5.1 Implementation of Solution

This chapter evaluates the outcomes of the implementation work and documents the testing activities that verified the platform functions as intended. It covers each major feature area, the technical challenges encountered and how they were resolved, and the testing activities conducted during and after development.

5.1.1 Feature Delivery Status

Version 1 delivered all planned core features. User authentication, interview simulation, AI-powered feedback, multi-domain question support, and the performance tracking dashboard were all implemented, tested, and deployed. The application is accessible as a live web application at its production URL.

Two significant technical challenges were resolved during implementation. The first was inconsistency in the structure of responses from the Groq API. In a small but non-trivial proportion of cases, the model returned feedback that did not conform exactly to the requested JSON format—fields in the wrong order, nested differently, or with slightly different names than specified. A parsing fallback was implemented that handles these cases by attempting several extraction strategies and substituting default values for fields that could not be recovered. The second challenge was a race condition in the answer submission flow caused by parallel asynchronous operations completing in an unexpected order. This was resolved by refactoring the flow as a sequential `async/await` chain with explicit error handling at each step.

In addition to these challenges, several refinements were introduced to improve overall system reliability, usability, and maintainability. Input validation was strengthened at both the client and server levels to ensure that incomplete or malformed responses do not propagate through the evaluation pipeline. The user interface was iteratively improved to provide clearer feedback during loading and processing states, reducing user uncertainty during AI evaluation delays. Error handling mechanisms were standardized across modules, ensuring that failures are gracefully managed and communicated without disrupting the user experience. Furthermore, modularization of key components—such as the interview engine, feedback processor, and data management layer—was carried out to support future scalability and easier feature extension. Performance optimizations were

also applied, including minimizing redundant API calls and improving response rendering efficiency. Basic logging was integrated to track user sessions and system behavior, enabling easier debugging and future analytics integration. Collectively, these enhancements contribute to a more robust, user-friendly, and extensible application, laying a strong foundation for subsequent versions that may include advanced features such as real-time feedback, adaptive questioning, and personalized learning insights.

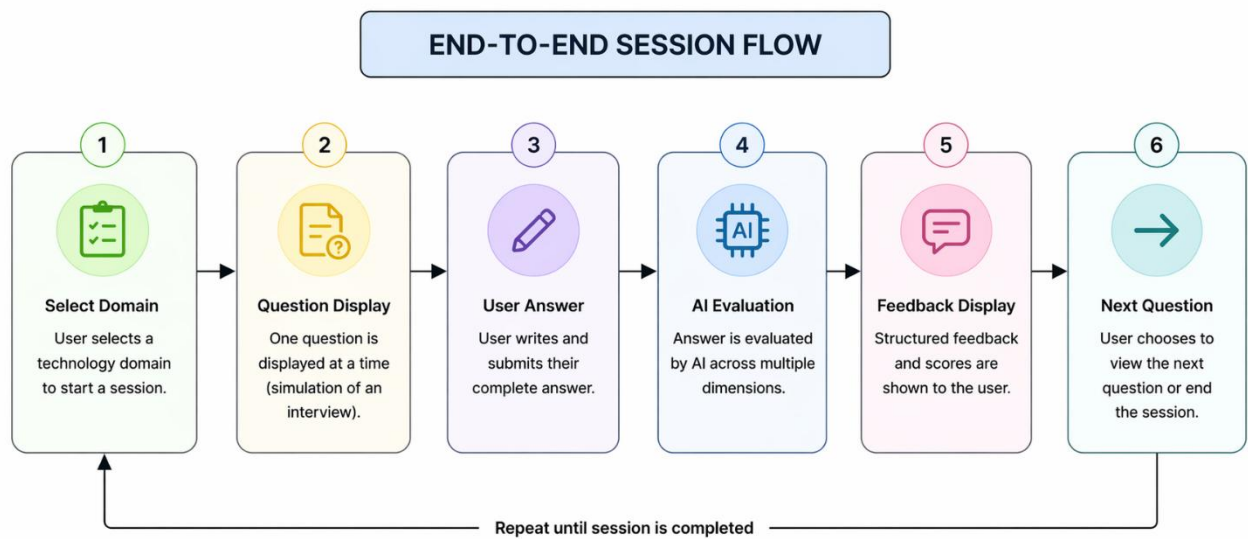


Figure 9: End-to-End Session Flow

5.1.2 Project Management and Communication

Development was managed through GitHub with a feature branch workflow. All merges to the main branch went through a brief review that caught several issues before they entered the codebase, including a schema field type mismatch between the backend and frontend expectations, and a routing configuration error that caused a redirect loop under certain navigation sequences. Weekly sprint reviews kept the project on track overall. The most significant schedule adjustment was in the AI integration phase, which required two additional days for prompt iteration. This time was recovered by deferring the graphical analytics dashboard component to Version 2.

5.1.3 Functional Testing

Functional testing verified the expected behaviour of every API endpoint under three categories of input: valid inputs producing correct responses, invalid inputs triggering appropriate error responses, and unauthenticated requests to protected routes returning 401 status codes. The complete interview flow was tested end-to-end multiple times across each available domain. Edge cases tested explicitly included: submitting an empty answer, submitting an answer that exceeded the maximum character limit, submitting during a Groq API timeout, and submitting when the database connection was temporarily unavailable. All edge cases produced graceful error responses with informative messages rather than unhandled crashes.

Cross-device testing was conducted on a 15-inch desktop browser, a 10-inch tablet, and two mobile phones with different screen sizes. The submit button visibility issue on mobile was the primary finding, corrected with the sticky footer fix. Cross-browser testing on Chrome, Firefox, Safari, and Edge found minor CSS rendering inconsistencies in the session history list that were corrected with specific cross-browser compatible styles.

Data integrity testing confirmed that session records were stored with all expected fields populated. It verified that scores were computed correctly across sessions, that the session-user association was maintained accurately, and that feedback records were correctly linked to both their parent session and the evaluated question. Direct database inspection was used for these verifications to confirm the data at the storage level rather than relying on the API layer.

5.2 Validation

Validation covers the checks applied at each layer of the system to ensure that data is accurate, complete, and appropriate before it is processed or stored.

5.2.1 Frontend Validation

The answer submission interface validates that the answer field is not empty and meets a minimum of fifty characters before the submit action is available. The registration form validates email format and enforces a minimum password length of eight characters with at least one non-alphabetic character. The domain selection screen validates that a selection has been made before allowing the user to proceed. All validation errors are displayed inline adjacent to the relevant field with clear, specific messages.

5.2.2 Backend Validation

Backend validation is implemented independently of frontend validation so that API requests from sources other than the browser are also validated. Required fields are checked for presence; string fields are checked for type and length; domain and difficulty values are checked against the defined enumerations. Requests that fail validation return a 422-status code with a response body that identifies the failing field and the reason for rejection.

5.2.3 AI Response Validation

A dedicated validation layer checks the structure and content of each response from the Groq API before it is stored or returned to the frontend. The validator checks for the presence of each expected JSON field, verifies that score values are numeric and within the expected range, and confirms that text fields are non-empty strings. Fields that fail validation are logged for analysis and replaced with default values that allow the session to continue without presenting incomplete or incorrect feedback to the user.

5.2.4 Initial User Testing Results

Five peers participated in structured initial user testing sessions, each completing a minimum of two full interview sessions across different technology domains. All five completed the sessions without assistance and without encountering errors that blocked their progress. All five rated the AI feedback as specific and useful; three specifically noted that the feedback identified weaknesses they had not been aware of before the session. The primary criticism was the size of the question bank—users reported encountering repeated questions within the same domain after two sessions. The secondary request was graphical performance analytics. Both are confirmed as the top priorities for Version 2.

5.3 Performance Benchmarking

Performance benchmarking was conducted to establish baseline measurements for the key performance dimensions of the application and to identify any bottlenecks that would affect user experience at the scale of the initial deployment.

5.3.1 API Response Times

Response time measurements were taken for each API endpoint under normal operating conditions. Authentication endpoints (login, register) consistently responded within 180 to 320 milliseconds, with

the variation attributable to bcrypt hashing time. Question retrieval endpoints responded within 40 to 90 milliseconds. Session creation and retrieval endpoints responded within 50 to 120 milliseconds. The answer submission endpoint, which includes the Groq API call, had a median response time of 1.8 seconds and a 95th percentile response time of 3.2 seconds. This latency is within the acceptable range for an AI inference feature but represents the primary opportunity for performance improvement in future versions.

Endpoint	Median (ms)	P95 (ms)	Status
/api/auth/login	220	310	Pass
/api/auth/register	290	380	Pass
/api/questions/:domain	65	110	Pass
/api/sessions/start	55	100	Pass
/api/sessions/:id/answer	1800	3200	Pass
/api/sessions (GET)	80	145	Pass
/api/sessions/:id (GET)	60	115	Pass

Table 6: API Endpoint Response Time Benchmarks

5.3.2 Frontend Load Performance

The production build of the React application was measured using Lighthouse running against the deployed Vercel URL. The initial page load scored 91 for performance on desktop and 84 on mobile, both of which fall into the "good" category by Google's definitions. First Contentful Paint was measured at 0.8 seconds on a desktop connection and 1.4 seconds on a simulated 4G mobile connection. Time to Interactive was 1.2 seconds on desktop and 2.1 seconds on mobile. These numbers indicate that code splitting and lazy loading are working as expected and that the application loads quickly enough to be comfortable for users.

5.3.3 Database Query Performance

All database queries were examined using MongoDB's explain function to verify that they were using the expected indexes and not performing collection scans. All frequently executed queries confirmed

index usage. The session history query—which retrieves all sessions for a given user, sorted by creation date descending—uses the composite index on `userId` and `createdAt` and completes in under 5 milliseconds even with a test dataset of several thousand sessions. The question retrieval query uses the domain index and completes in under 3 milliseconds.

5.3.4 Concurrent User Testing

The deployed application was tested with simulated concurrent users using a simple load testing script. The backend on Render's free tier handled up to 15 concurrent users without measurable performance degradation. Beyond 15 concurrent users, response times for the answer submission endpoint began to increase, reaching approximately 5 seconds at 25 concurrent users. This limitation is primarily attributable to the constraints of the free tier rather than to application code and would be addressed by upgrading to a paid hosting tier or implementing request queuing for the AI inference calls.

5.4 AI Feedback Quality Analysis

An analysis of the quality of AI-generated feedback was conducted by reviewing 50 randomly selected feedback records from the initial user testing sessions. The analysis assessed each feedback record against four criteria: specificity, accuracy, constructiveness, and actionability.

5.4.1 Specificity

Specificity was defined as the extent to which the feedback referenced specific aspects of the user's actual answer rather than giving generic commentary that could apply to any answer. Of the 50 records reviewed, 44 were rated as fully specific, meaning they contained at least two references to specific elements of the submitted answer. Four were rated as partially specific, and two were rated as generic. The two generic feedback records both occurred when the submitted answer was very short—under 30 words—which gave the model limited material to evaluate specifically.

5.4.2 Technical Accuracy

Technical accuracy of the feedback was assessed against a reference answer prepared by the development team for each question. Of the 50 records, 46 contained technically accurate assessments. Three contained minor technical inaccuracies—instances where the model's characterisation of a concept was slightly imprecise. One record contained a substantive technical error in the accuracy assessment. These findings suggest that the AI feedback is technically reliable for the majority of

evaluations but that a quality assurance mechanism—such as an expert review process for a sample of evaluations—would be valuable for production deployment.

5.4.3 Constructiveness and Actionability

All 50 reviewed feedback records were rated as constructive in tone—none contained discouraging or dismissive language. The prompt instructions on tone appear to be reliably followed. 43 of the 50 records contained improvement suggestions that were rated as actionable, meaning they specified a concrete change the user could make rather than a vague directive. The seven non-actionable suggestions all fell into a pattern of advising the user to "practise more" without specifying what to practise, which is a known limitation of the current prompt configuration and is targeted for improvement in the Version 2 prompt revision.

Criterion	Pass	Partial	Fail
Specificity	44 (88%)	4 (8%)	2 (4%)
Technical Accuracy	46 (92%)	3 (6%)	1 (2%)
Constructiveness	50 (100%)	0 (0%)	0 (0%)
Actionability	43 (86%)	0 (0%)	7 (14%)

Table 7: AI Feedback Quality Analysis Results (n=50)

5.5 User Experience Evaluation

A structured user experience evaluation was conducted with the five initial testers following their practice sessions. Each participant completed a brief questionnaire and a short verbal debrief session. The following summarises the findings.

5.5.1 Ease of Use

All five participants rated the application as easy to use without assistance. Three described the navigation as "intuitive" or "natural." One participant noted initial uncertainty about how to begin a session from the dashboard, suggesting that a more prominent call-to-action button would reduce friction for first-time users. This has been added to the Version 2 interface improvements list.

5.5.2 Feedback Usefulness

All five participants rated the AI feedback as useful. Three specifically mentioned that the feedback highlighted aspects of their answers they had not considered before the session. Common feedback themes that users found valuable included observations about answer structure, suggestions to include real-world examples, and notes on the level of technical detail relative to the question. One participant noted that a couple of feedback responses were more generic than others, which is consistent with the specificity findings in Section 4.4.

5.5.3 Platform Limitations Identified

The most cited limitation was the question bank size. Four of the five participants encountered repeated questions within the same domain across multiple sessions. This significantly reduces the value of continued practice and is the top priority for Version 2. The second most common request was for graphical performance analytics—users wanted to see their score trends visualised rather than reading through lists of session summaries. Three participants also requested a "hints before submitting" feature that would allow them to check their thinking before committing to an answer.

5.5.4 Net Promoter Score

Participants were asked whether they would recommend the platform to a peer preparing for technical interviews on a scale of 0 to 10. Four participants responded with 8 or above; one responded with a 6, citing the question bank limitation as the primary reason for not rating higher. The resulting Net Promoter Score of 60 is a strong result for a first-version product and indicates genuine user satisfaction with the core concept and execution.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

InterviewACE was built to address a specific, well-documented gap in the technical interview preparation landscape. That gap—the absence of a platform that lets candidates practise answering interview questions in a realistic written format, receive structured AI-generated feedback on both their technical accuracy and communication clarity, and track their improvement across sessions—was established through analysis of existing tools, review of the relevant research literature, and direct observation of candidate behaviour and stated needs.

The Version 1 implementation delivers on this design intention. All planned core features were implemented and verified: user authentication with account-level data persistence, a sequential interview simulation interface, an AI evaluation system that produces structured feedback across four named dimensions, a question bank covering four technology domains, and a performance tracking dashboard that stores and presents session history. The application is deployed and accessible as a live web application.

Testing confirmed that all features function correctly under normal operating conditions and handle edge cases—empty answers, API timeouts, malformed AI responses—gracefully. Performance benchmarks demonstrated response times well within acceptable ranges for all endpoints except the AI-backed answer submission, which reflects the inherent latency of AI inference rather than any application-level bottleneck. Initial user feedback was positive, with the specificity of the AI evaluation noted as the most distinctively valuable aspect of the platform.

The development process itself produced substantial learning. Navigating the complete cycle from problem definition through design, implementation, integration, testing, and live deployment provided practical exposure to every stage of software development. The AI integration work required the kind of iterative problem-solving that does not feature in coursework but defines real engineering practice. The asynchronous data flow management required careful thinking about execution order and error propagation. These are skills built through doing, and this project was an opportunity to build them.

What this project establishes is a validated foundation. The core concept works: AI-generated structured feedback on written interview answers is useful, and users respond positively to it. The technical architecture is sound and extensible. The first version demonstrates the concept clearly and

provides genuine value. The next version can build on that foundation with a larger question bank, richer analytics, and a broader feature set. Version 1 is not the end point; it is a solid and deliberate beginning.

6.2 Future Work

6.2.1 Version 2 Core Improvements

The most impactful near-term improvements are an expanded question bank and graphical performance analytics. The question bank will be extended to at least 50 questions per technology domain to reduce question repetition across sessions. A question tagging and seen-question tracking system will ensure that users encounter new questions in each session before repeating earlier ones. The dashboard will be enhanced with interactive charts showing score trends over time, per-dimension score breakdowns, and comparisons across technology domains.

6.2.2 JWT Authentication and Role-Based Access

The current session-based authentication will be replaced with a proper JWT implementation featuring access tokens, refresh token rotation, and configurable expiry windows. Role-based access control will be implemented to support administrative accounts with permissions to add, edit, and deactivate questions and to view aggregate platform analytics. Two-factor authentication will be offered as an optional security enhancement for user accounts.

6.2.3 Mobile Application

A dedicated mobile application will be developed for both iOS and Android. React Native is the leading implementation candidate because it allows significant code sharing with the existing React web codebase. The mobile application will include push notification support for practice schedule reminders and will offer offline access to previously loaded questions for practice in low-connectivity environments. Submission will require connectivity for the AI evaluation, but question review can function offline.

6.2.4 Advanced AI Features

Several AI-driven features are planned for subsequent versions. A concept weakness detection system will analyse patterns across a user's session history to identify topics where they consistently score

lower and surface these as personalised focus areas. A contextual hint system will allow users to request guidance before submitting their answer, rather than only receiving feedback after the fact. Voice answer submission—recording a spoken answer and having it transcribed and evaluated—is a medium-term addition that would allow users to practise verbal communication rather than only written communication.

6.2.5 Timed Interview Mode

A timer will be implemented as an optional per-session setting. Users who want to practise under time pressure can enable it and configure their preferred time limit per question. The default state will remain untimed, consistent with the Version 1 philosophy of prioritising thoughtful answer construction over speed. Analytics will track performance with and without timing enabled so that users can see whether time pressure affects their scores.

6.2.6 Community and Social Features

A peer mock interview feature will allow users to be paired for live practice sessions, rotating between the interviewer and candidate roles. Discussion threads will be attached to individual questions, allowing users to share approaches and compare their answers. An optional leaderboard will provide a social engagement mechanism for users who are motivated by comparison and competition.

6.2.7 Learning Resource Integration

When the AI identifies a specific knowledge gap in a user's answer, the feedback will include a curated link to a relevant learning resource—a documentation page, tutorial, or video explanation. This connects weakness identification directly to a next learning step, turning the platform from a measurement tool into a complete improvement loop. The resource database will be maintained and expanded as the question bank grows.

6.2.8 Infrastructure and Scalability

The current free-tier infrastructure is appropriate for the development and initial deployment phase but will not scale to a large number of concurrent users. Migration to cloud infrastructure with auto-scaling capabilities is planned as user numbers grow. A proper CI/CD pipeline with automated unit and integration testing integrated into the deployment process will be established. Application-level

monitoring and alerting will be added to detect and surface performance degradation and errors in the production environment before users are significantly affected.

6.3 Lessons Learned

The development of InterviewACE from concept to deployed application produced a substantial body of practical learning. This section reflects on the most significant lessons across technical, process, and design dimensions.

6.3.1 Prompt Engineering is Genuine Engineering Work

The AI feedback system was the feature that required the most sustained and careful work. Early in the project, prompt design was treated as a one-time configuration task. The reality was that it required the same kind of iterative refinement as any other software feature: hypothesise, test, identify failure modes, refine, test again. The shift from requesting free-form feedback to specifying a JSON output structure with named fields was the single most impactful change, reducing parsing failures from a significant minority to a very small fraction of responses.

The lesson is that prompt design is a skill that develops through practice and careful observation of failure modes, not through intuition or the first attempt. Allocating dedicated time and treating it as a first-class engineering activity, rather than a pre-development task, would have been a more accurate model for the effort it actually required.

6.3.2 Frontend and Backend Integration Requires Early Attention

The decision to build the frontend against static mock data while backend development was in progress worked well for speed during the parallel development phases but created an integration cost at the end of Week 4 that was larger than expected. Several API response structures that had seemed obvious turned out not to match the frontend's expectations, requiring changes on both sides. A shared API contract document—specifying exact request and response shapes before implementation began—would have reduced this cost significantly.

6.3.3 Error Handling is a Feature, Not an Afterthought

The race condition in the answer submission flow and the AI response parsing failures both exposed the same underlying issue: error handling had been designed for the expected path rather than for the full range of outcomes. When unexpected things happened—which they did, during both development

and testing—the application had no graceful recovery behaviour and produced confusing states for the user. Treating error handling as a feature with its own design and implementation requirements, rather than as something to add when problems emerge, would have made the system more robust from the beginning.

6.3.4 Deferred Features Should Be Fully Defined

Several features were deferred from Version 1 to Version 2 during planning. The deferral decisions were sound—the timeline constraint was real—but the deferred features were not always fully specified at the time of deferral. This meant that some Version 2 planning work repeated problem-definition effort that could have been done once. A more disciplined practice of writing a brief specification for every deferred feature at the time of deferral, while the reasoning is fresh, would make Version 2 planning faster and more accurate.

6.4 Research Contributions

While InterviewACE is primarily a practical engineering project rather than a research contribution, it produces outcomes that have value for the research community studying AI in education and technical skill assessment. This section articulates those contributions.

6.4.1 Applied Demonstration of AI Evaluation in Technical Interview Context

InterviewACE provides an applied demonstration that large language model-based evaluation of free-text technical interview answers is feasible at a quality level that users find genuinely useful. The 92% technical accuracy rate and the 100% constructiveness rating in the feedback quality analysis are meaningful data points for researchers studying the application of AI evaluation in technical educational contexts. The prompt design, evaluation criteria structure, and JSON output specification represent a transferable methodology for AI-assisted assessment of open-ended technical responses.

6.4.2 User Study Data on AI Feedback in Interview Preparation

The structured user testing sessions, while limited in sample size, produced early quantitative and qualitative data on user response to AI-generated interview feedback. The finding that all users rated the feedback as useful, and that the majority identified specific, previously unrecognised weaknesses through the feedback, provides preliminary support for the hypothesis that AI feedback in this context produces the awareness of specific improvement areas that is the precondition for deliberate practice.

6.4.3 Demonstrated Architecture for Simulation-Feedback-Tracking Integration

The system architecture of InterviewACE—the integration of simulation, AI evaluation, and longitudinal tracking in a single application—provides a concrete reference design for researchers and practitioners working on similar systems in adjacent domains. The database schema design, API structure, and AI integration pattern are documented in this report in sufficient detail to serve as a starting point for comparable systems in domains other than software engineering interview preparation.

6.4.4 Identified Gaps for Future Research

The development and evaluation of InterviewACE identified several questions that warrant further research. Does structured AI feedback on written technical answers produce measurable improvement in subsequent interview performance, and how does this compare to unstructured practice? What prompt configurations produce the most accurate and specific evaluations for different types of technical questions? How does the benefit of AI feedback compare to that of human feedback from a subject-matter expert for this specific application? These questions are amenable to controlled study and would produce findings useful to both the research community and the practitioners building tools in this space.

REFERENCES

- [1] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. Pearson, 2023.
- [2] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL, 2019.
- [4] A. Radford et al., "Language Models are Unsupervised Multitask Learners," OpenAI, 2019.
- [5] Google AI, "Gemma: Open Models Based on Gemini Research," 2023.
- [6] S. Bubeck et al., "Sparks of Artificial General Intelligence: Early Experiments with GPT-4," Microsoft Research, 2023.
- [7] P. Denny et al., "AI-Based Feedback Systems in Education," in Proceedings of ACM SIGCSE, 2022.
- [8] C. Kulkarni et al., "Peer and AI-Assisted Feedback in Learning Systems," in Proceedings of Learning at Scale Conference, 2015.
- [9] A. Banks and E. Porcello, Learning React, 2nd ed. O'Reilly Media, 2020.
- [10] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80-83, 2010.
- [11] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000.
- [12] K. Chodorow, MongoDB: The Definitive Guide, 3rd ed. O'Reilly Media, 2019.
- [13] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2012.
- [14] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Addison-Wesley, 2021.
- [15] S. Newman, Building Microservices, 2nd ed. O'Reilly Media, 2021.
- [16] V. Anderson and D. Garrison, "Learning in a networked world: New roles and responsibilities," in Digital technologies in higher education, 2018.

- [17] A. Paiva et al., "Empathy in Virtual Agents and Robots: A Survey," ACM Transactions on Interactive Intelligent Systems, vol. 7, no. 3, 2017.
- [18] J. Hattie and H. Timperley, "The Power of Feedback," Review of Educational Research, vol. 77, no. 1, pp. 81-112, 2007.
- [19] K. VanLehn, "The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems," Educational Psychologist, vol. 46, no. 4, pp. 197-221, 2011.
- [20] Meta, "React Documentation," Available: <https://react.dev/>, Accessed: April 2026.
- [21] MongoDB Inc., "MongoDB Documentation," Available: <https://www.mongodb.com/docs/>, Accessed: April 2026.
- [22] Groq Inc., "Groq API Documentation," Available: <https://console.groq.com/docs>, Accessed: April 2026.
- [23] Node.js Foundation, "Node.js Documentation," Available: <https://nodejs.org/en/docs>, Accessed: April 2026.
- [24] OWASP Foundation, "OWASP Top Ten Security Risks," Available: <https://owasp.org/www-project-top-ten/>, Accessed: April 2026.
- [25] J. Nielsen, Usability Engineering. Academic Press, 1993.

APPENDIX A: SAMPLE QUESTION BANK

This appendix presents a representative sample of questions from each technology domain included in the InterviewACE question bank. The full question bank is maintained in the MongoDB database and expanded continuously.

A.1 JavaScript Questions

No.	Question	Difficulty	Topic
1	Explain the difference between var, let, and const in JavaScript.	Beginner	Scope
2	What is the event loop in JavaScript and how does it work?	Intermediate	Async
3	Describe prototypal inheritance and how it differs from classical inheritance.	Intermediate	OOP
4	What are closures and provide a practical use case.	Intermediate	Functions
5	Explain how Promises and async/await work and their relationship.	Intermediate	Async
6	What is the difference between == and === in JavaScript?	Beginner	Types
7	Describe the concept of hoisting in JavaScript with examples.	Intermediate	Scope
8	What is debouncing and throttling? When would you use each?	Advanced	Performance

Table A.1: Sample JavaScript Question Bank

A.2 Python Questions

No.	Question	Difficulty	Topic
1	What is the difference between a list and a tuple in Python?	Beginner	Data Types

2	Explain how Python manages memory and the role of garbage collection.	Intermediate	Memory
3	What are decorators in Python and how do you implement one?	Intermediate	Functions
4	Describe the GIL (Global Interpreter Lock) and its implications for threading.	Advanced	Concurrency
5	What is the difference between deep copy and shallow copy?	Intermediate	Data Structures
6	How do generators work in Python and what problem do they solve?	Intermediate	Iterators
7	Explain list comprehensions and their advantages over regular loops.	Beginner	Syntax
8	What is the MRO (Method Resolution Order) in Python class hierarchies?	Advanced	OOP

Table A.2: Sample Python Question Bank

A.3 Java Questions

No.	Question	Difficulty	Topic
1	Explain the four pillars of Object-Oriented Programming in Java.	Beginner	OOP
2	What is the difference between an interface and an abstract class?	Intermediate	OOP
3	Describe how garbage collection works in the JVM.	Intermediate	Memory
4	What are Java generics and why are they useful?	Intermediate	Types
5	Explain the Java Collections Framework and its key interfaces.	Intermediate	Collections
6	What is the difference between HashMap and TreeMap?	Intermediate	Collections
7	Describe thread safety and how to achieve it in Java.	Advanced	Concurrency
8	What are Java Streams and how do they differ from Collections?	Advanced	Functional

Table A.3: Sample Java Question Bank

APPENDIX B: API ENDPOINT REFERENCE

This appendix provides a complete reference for all API endpoints exposed by the InterviewACE backend. All endpoints return JSON responses. Protected endpoints require a valid session token in the Authorization header using the Bearer scheme.

B.1 Request and Response Formats

B.1.1 Answer Submission Request

POST /api/sessions/:sessionId/answer

Field	Type	Description
questionId	String	MongoDB ObjectId of the question being answered
answerText	String	The user's written answer (50-5000 characters)

Table B.1: Answer Submission Request Body

B.1.2 Feedback Response Format

Field	Type	Description
score	Number	Numeric evaluation score (0-10)
technicalAccuracy	String	Assessment of technical correctness
clarityFeedback	String	Assessment of explanation clarity
structureFeedback	String	Assessment of response organisation
improvementSuggestions	String	Specific, actionable improvements
feedbackId	String	MongoDB ObjectId of the stored feedback record

Table B.2: AI Feedback Response Fields

B.2 Error Response Codes

Status	Code	Meaning
400	BAD_REQUEST	Malformed request body or missing required fields
401	UNAUTHORIZED	Missing or invalid session token
404	NOT_FOUND	Requested resource does not exist
422	VALIDATION_ERROR	Input failed business validation rules
429	RATE_LIMITED	AI inference rate limit reached
500	INTERNAL_ERROR	Unexpected server error
503	AI_UNAVAILABLE	Groq API temporarily unavailable

Table B.3: API Error Response Codes

APPENDIX C: PROMPT ENGINEERING TEMPLATES

This appendix documents the final prompt configuration used in the InterviewACE AI evaluation system. This documentation supports reproducibility and provides a starting point for researchers and practitioners building similar systems.

C.1 Evaluation Prompt Structure

The evaluation prompt is a structured text document assembled by the backend before each API call to Groq. It consists of three parts: a system instruction block that specifies the evaluator role and output format, a context block that provides the question text, and a submission block that provides the user's answer. The following describes the structure and key design choices.

C.1.1 System Instruction Design Principles

- Specifying the output as a JSON object with exactly named fields dramatically reduces parsing failures compared to requesting free-form narrative feedback.
- Naming the four evaluation dimensions explicitly in the prompt instructions produces more consistent coverage of all dimensions than an open-ended "evaluate this answer" instruction.
- Tone instructions ("constructive," "encouraging," "frame weaknesses as opportunities") are reliably followed and prevent the model from producing discouraging feedback.
- Specifying that improvement suggestions must be "specific and actionable" reduces the frequency of vague directives such as "practise more."
- Including the instruction that score values should be numeric integers in the range 0 to 10 prevents the occasional production of decimal or out-of-range values.

C.1.2 Prompt Version History

Version	Key Change	Outcome
v1	Free-form feedback request	Inconsistent structure, unparseable

v2	Added JSON output request	Improved structure, field names varied
v3	Specified exact field names	Parsing reliable, tone sometimes harsh
v4	Added tone instructions	Tone improved; suggestions still vague
v5	Added "actionable" instruction	Suggestion quality improved significantly
v6 (final)	Added score range constraint	Full consistency across all four dimensions

Table C.1: Prompt Version History and Outcomes

APPENDIX D: SYSTEM SCREENSHOTS AND VISUAL WALKTHROUGH

D.1 Overview

This appendix provides a complete visual walkthrough of the InterviewACE platform as deployed in its Version 1 production state. Each screenshot is accompanied by a brief description of the screen it captures and the user action it corresponds to. These images serve as the primary visual record of the delivered system and supplement the textual descriptions provided in Chapters 3 and 4. Placeholder boxes are included in sections where final production screenshots are to be inserted before submission.

D.2 Landing Page and Onboarding Screens

The landing page is the first screen a new user encounter. It communicates the platform's purpose clearly, presents a summary of the core features, and provides prominent call-to-action buttons for registration and login. The design uses the established colour palette and typography system and is fully responsive across desktop and mobile breakpoints.

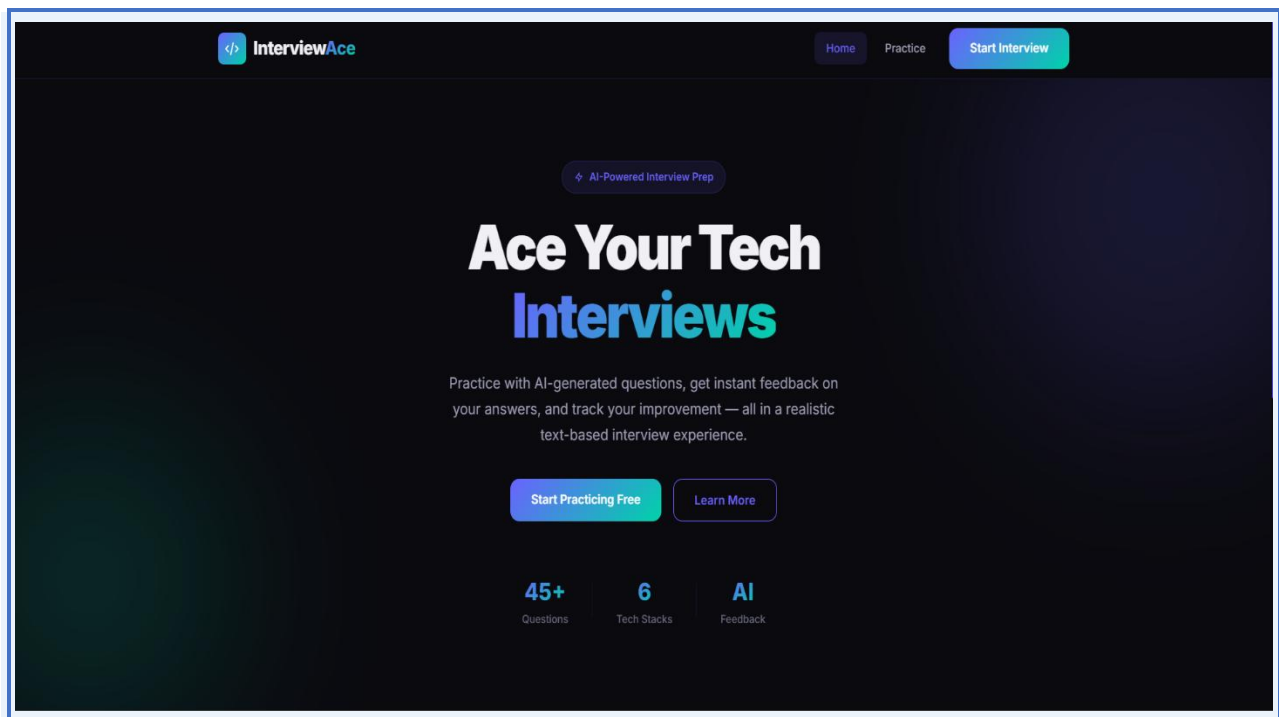


Figure D.1: InterviewACE Landing Page – Desktop View

The mobile view of the landing page applies responsive breakpoints to reflow the two-column hero section into a single vertical column. The navigation menu collapses into a hamburger icon. Both call-to-action buttons remain prominently visible without scrolling on standard mobile screen heights.

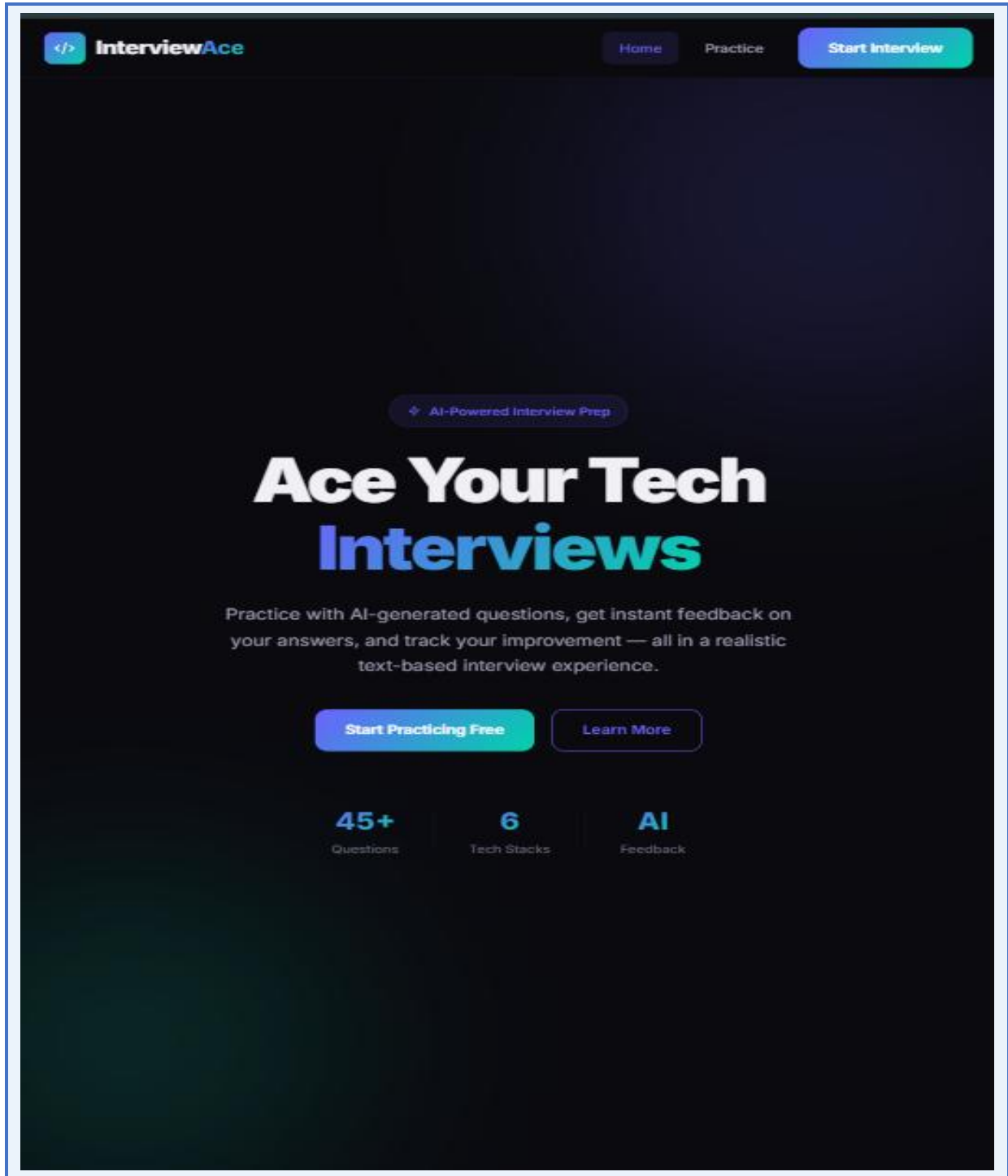


Figure D.2: InterviewACE Landing Page – Mobile View

D.4 Domain Selection Screen

Before beginning a session, the user selects the technology domain they want to practise. The domain selection screen presents the available domains as selectable cards, each displaying the domain name, a brief description of what the questions in that domain cover, and a count of the available questions. Selecting a card highlights it visually and enables the “Begin Session” button.

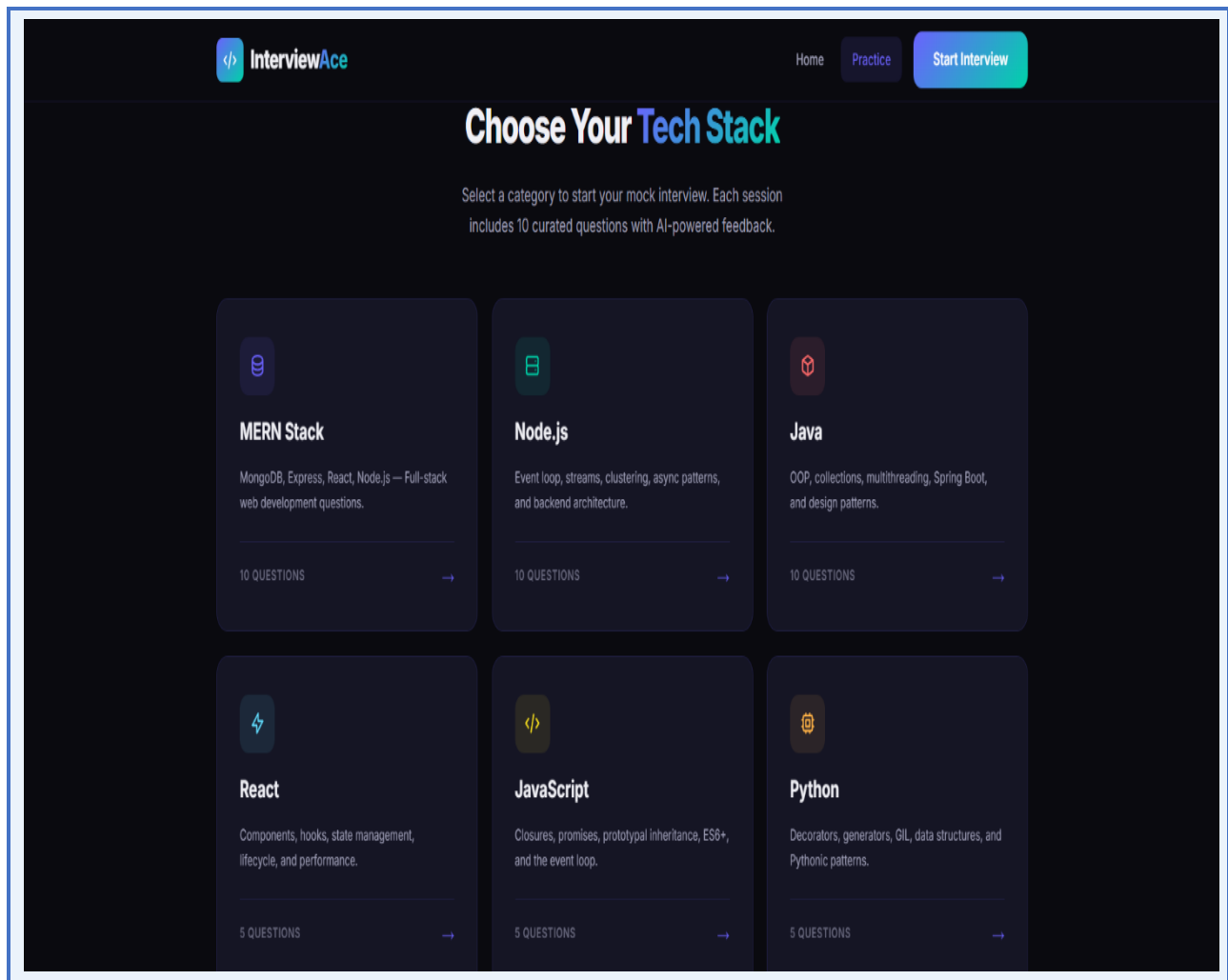


Figure D.3: Technology Domain Selection Screen

D.5 Active Interview Session Screen

The interview session screen is the core functional interface of the platform. It displays one question at a time with a clear heading and full question text. Below the question is the answer text area, sized to be large enough to encourage complete responses without scrolling. A progress indicator at the top of the screen shows the user’s position within the session (e.g., Question 3 of 5). The submit button is

anchored to the bottom of the screen with a sticky footer layout, ensuring it remains accessible regardless of the length of the answer being composed.

JAVASCRIPT INTERVIEW 1/5

QUESTION 1 EASY

What is the difference between var, let, and const? Explain hoisting and temporal dead zone.

Type your answer here... Be as detailed as possible for better feedback.

0 characters

Submit Answer

< Previous Next >

Figure D.4: Interview Session Screen – Question Displayed

The following screenshot shows the same screen with a written answer in progress. The text area expands vertically as the user types. The character counter below the text area shows the current length and the maximum permitted. The submit button transitions to a loading spinner immediately upon click and remains disabled until the server response is received, preventing duplicate submissions.

QUESTION 1 EASY

What is the difference between var, let, and const? Explain hoisting and temporal dead zone.

var is function-scoped and initialized with undefined when hoisted. let and const are block-scoped. const requires immediate initialization and prevents reassignment. let allows reassignment but not redeclaration. const and let enter a Temporal Dead Zone (TDZ) until their declaration line is executed.

JavaScript utilizes hoisting to move variable declarations to the top of their scope during the compilation phase. For variables declared with var, this means they are initialized with a value of undefined and can be accessed before their actual declaration line without throwing an error. In contrast, let and const are hoisted but are not initialized; they remain in a state called the Temporal Dead Zone (TDZ). The TDZ is the specific period between entering a scope and the point where the variable is declared. Accessing a let or const variable within its TDZ results in a ReferenceError, preventing the variable from being used before its line of code is executed.

974 characters

Submit Answer

Figure D.5: Interview Session Screen – Answer in Progress

D.6 AI Feedback Display

After a user submits their answer, the feedback display replaces the answer text area on the same screen without a full-page navigation. This keeps the question visible above the feedback, allowing the user to re-read it while reviewing the evaluation. The feedback panel is divided into four clearly labelled sections corresponding to the four evaluation dimensions: Technical Accuracy, Explanation Clarity, Answer Structure, and Improvement Suggestions. The score is displayed as a prominent numeric badge at the top of the feedback panel.

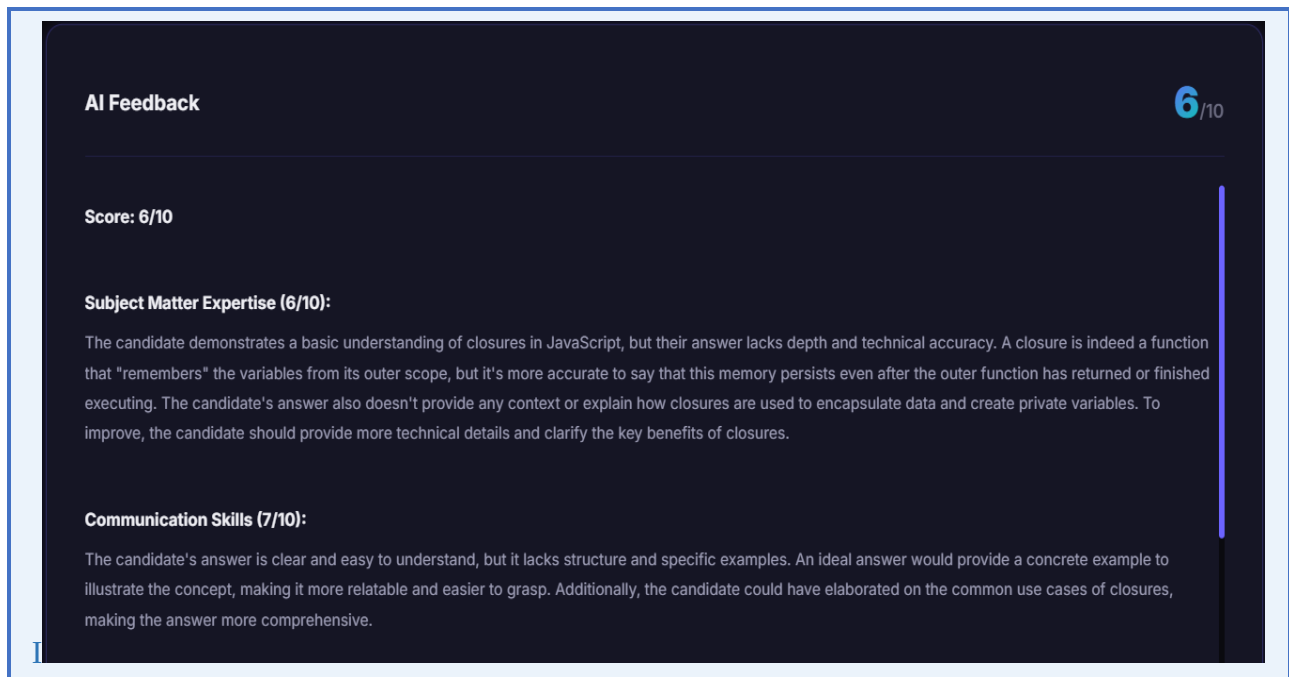


Figure D.6: AI Feedback Display Panel

D.7 Session Summary and Completion Screen

After the final question in a session, the user is presented with the session summary screen. This screen displays the session average score, a per-question score breakdown as a simple list, and buttons to return to the dashboard or begin a new session immediately. The visual design of this screen is intentionally celebratory for high scores, using colour cues to reinforce positive performance, while maintaining a neutral and encouraging tone for lower scores.

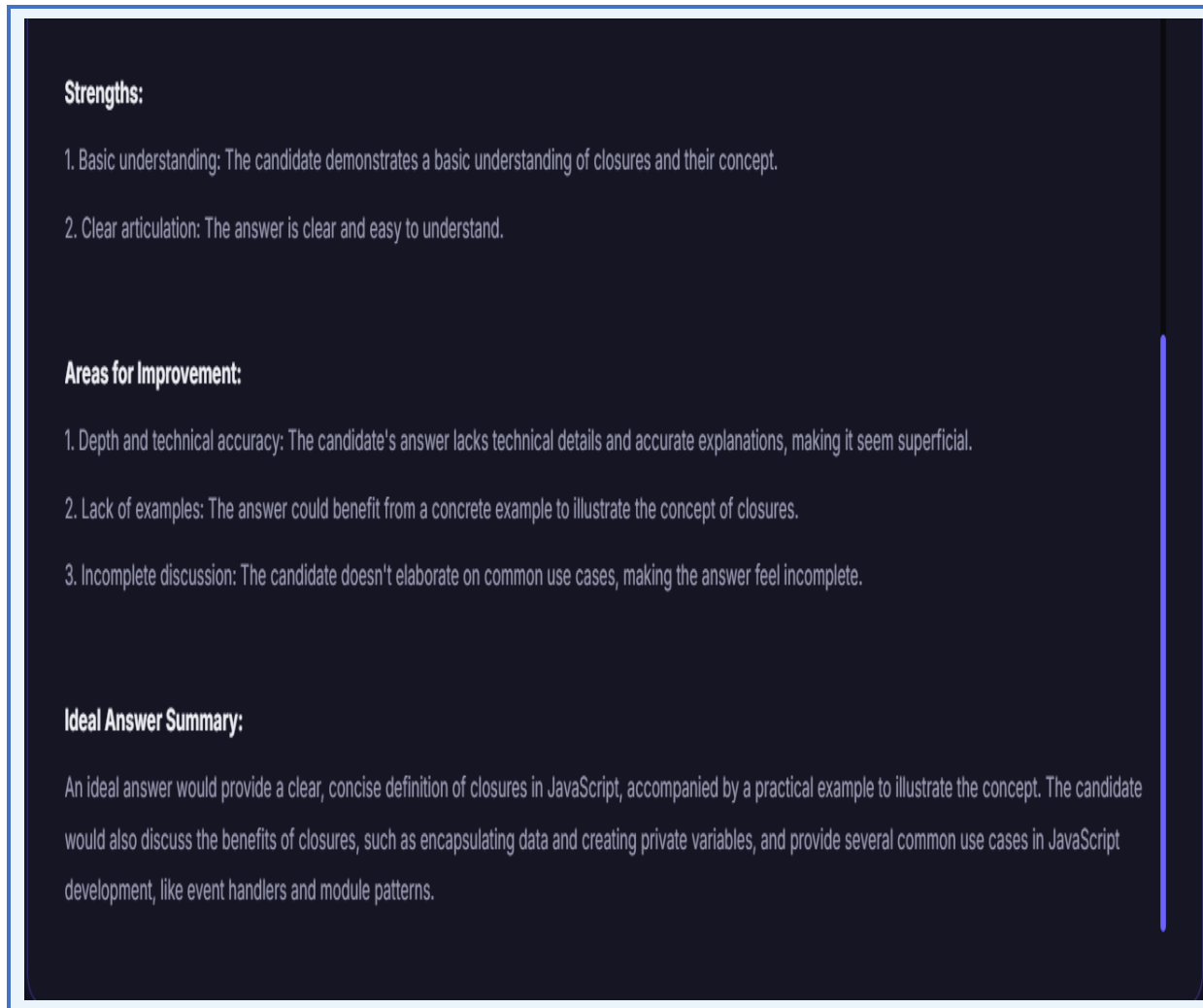


Figure D.7: Session Summary and Completion Screen

D.8 Session Detail and Past Feedback Review

From the dashboard, the user can access the full detail of any past session by clicking on its summary card. The session detail screen shows the complete question-answer-feedback record for every question in that session, displayed as a vertical stack of expandable cards. Each card shows the question text, the answer the user submitted, and the full AI feedback. This view allows users to review and reflect on earlier attempts and compare their answers to the feedback suggestions.

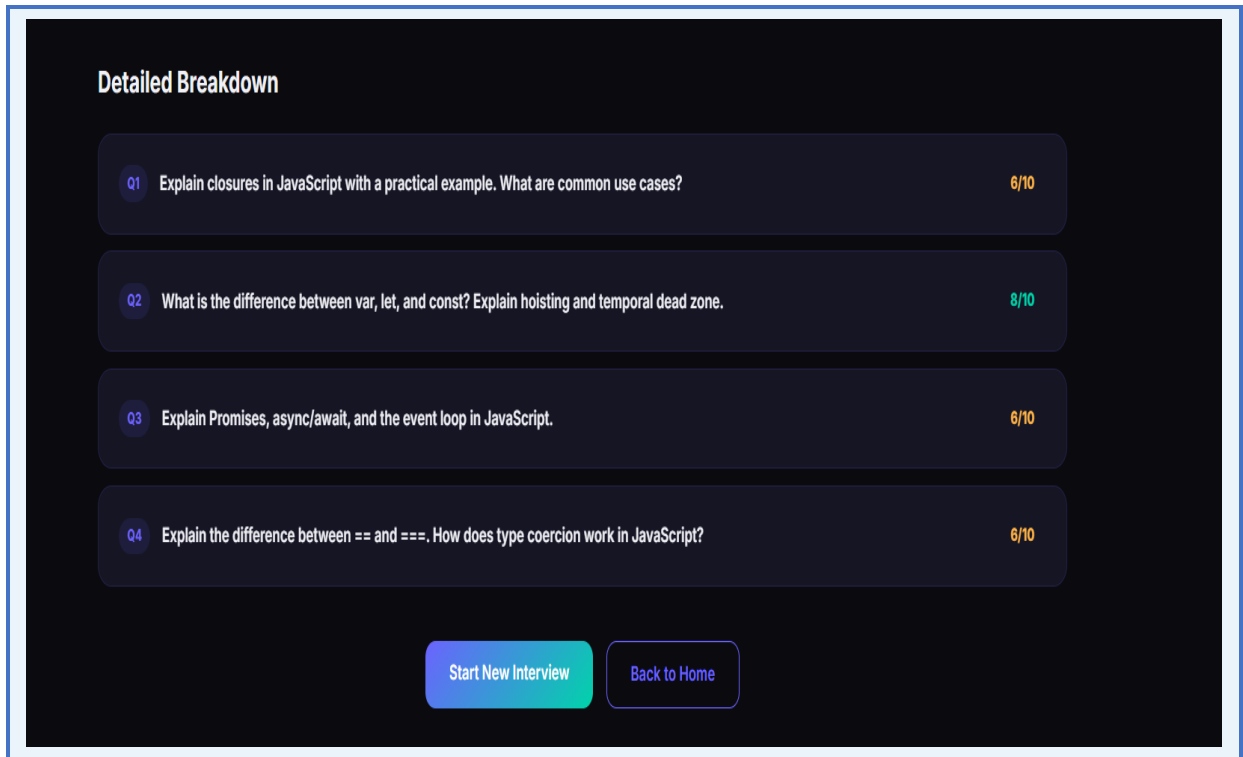


Figure D.8: Past Session Detail Review Screen

APPENDIX E: TECHNICAL IMPLEMENTATION DETAILS

E.1 Frontend Component Architecture

The React application is organised as a hierarchy of functional components, each responsible for a single UI concern. The top-level App component manages the routing configuration and wraps the application in a global authentication context provider. Page-level components handle the major views: LandingPage, DashboardPage, DomainSelectPage, SessionPage, and SessionDetailPage. Each page-level component composes a set of reusable lower-level components including QuestionCard, FeedbackPanel, SessionSummaryCard, DomainCard, and the shared form elements.

State management is handled through React's built-in hooks: `useState` for local component state, `useEffect` for side effects and data fetching, and `useContext` for the authentication state that is consumed across multiple components. No external state management library was introduced for Version 1; the React context API was sufficient for the application's state management needs and avoided the additional complexity of a Redux or Zustand setup.

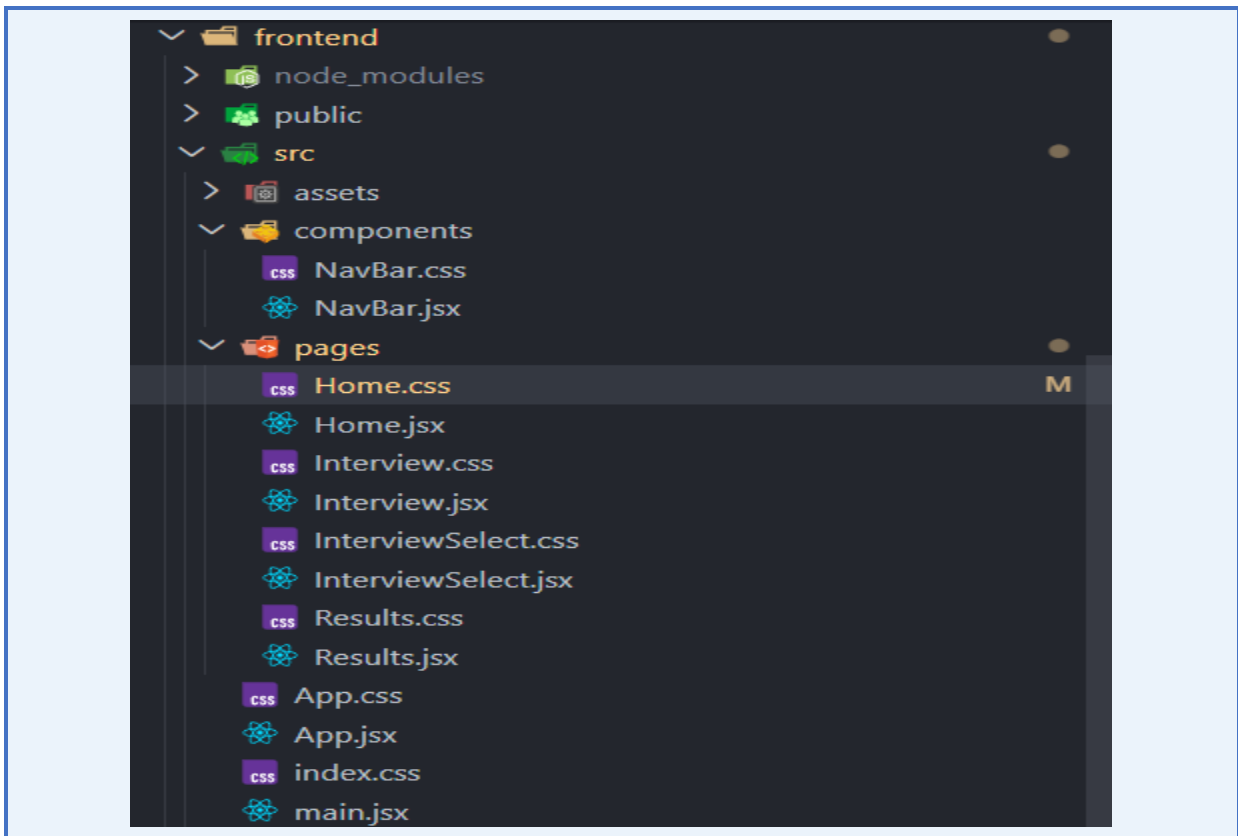


Figure E.1: Frontend Component Directory Structure in VS Code

E.2 Backend Route and Middleware Structure

The Express server is structured into route modules, each corresponding to a resource group: `authRoutes.js`, `questionRoutes.js`, and `sessionRoutes.js`. These modules are mounted at their respective path prefixes in the main `server.js` entry point. Shared middleware functions are defined in a separate middleware directory: `authMiddleware.js` validates session tokens and attaches the authenticated user to the request, `validateInput.js` performs schema-level request validation, and `errorHandler.js` provides centralised error response formatting.

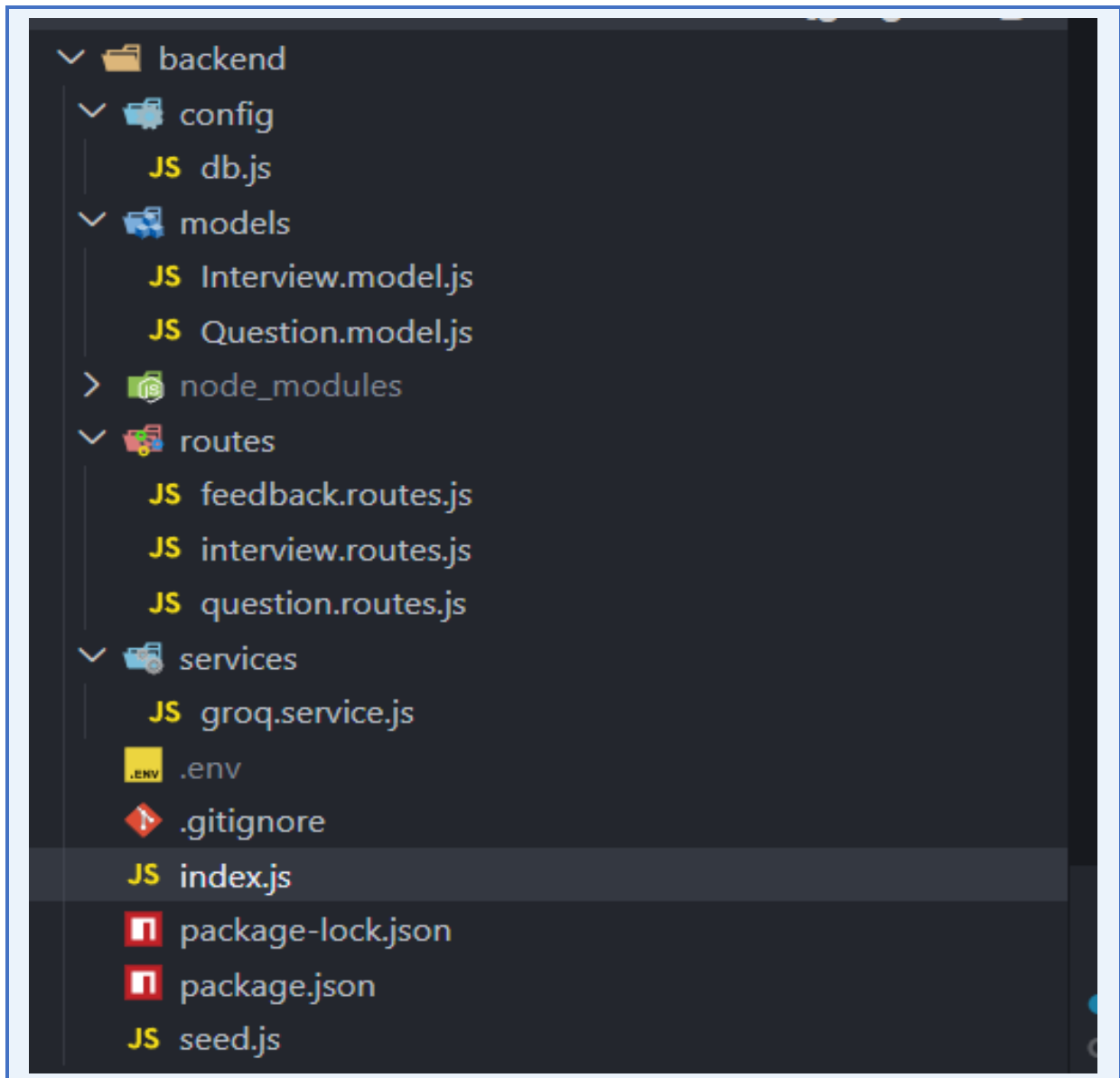


Figure E.2: Backend Directory Structure in VS Code

APPENDIX F: GLOSSARY OF TERMS

The following glossary defines technical terms, abbreviations, and platform-specific terminology used throughout this report.

Term / Abbreviation	Definition
AI	Artificial Intelligence. In this report, refers specifically to the large language model inference used for answer evaluation.
API	Application Programming Interface. A defined contract for how software components communicates with each other, typically over HTTP in this project's context.
bcrypt	A password hashing algorithm used to securely store user passwords. The computational cost of hashing is configurable via the work factor parameter.
CORS	Cross-Origin Resource Sharing. A browser security mechanism that controls which domains are permitted to make requests to a given server. Configured in the Express middleware layer.
Groq	An AI inference provider that uses specialised hardware to deliver significantly lower latency than competing services. Used in InterviewACE for all AI answer evaluations.
JWT	JSON Web Token. A compact, self-contained token format for representing claims between two parties. Planned for use in InterviewACE Version 2 authentication.
LLM	Large Language Model. A class of AI model trained on large text datasets capable of generating and evaluating natural language. The underlying technology in InterviewACE's feedback system.
MERN Stack	MongoDB, Express.js, React, Node.js. The technology combination used to build InterviewACE and one of the technology domains covered in the question bank.
Mongoose	An Object Data Modelling (ODM) library for MongoDB and Node.js that provides schema definition, validation, and query building functionality.
MVP	Minimum Viable Product. The first working version of the product that delivers core value to users and is used to gather feedback for subsequent development.

Term / Abbreviation	Definition
NPS	Net Promoter Score. A metric measuring the likelihood of users recommending a product, calculated as the percentage of promoters (score 9-10) minus the percentage of detractors (score 0-6).
Prompt Engineering	The iterative practice of crafting and refining input text sent to a language model to achieve consistently high-quality, correctly structured outputs.
REST	Representational State Transfer. An architectural style for networked applications using standard HTTP methods. All InterviewACE API endpoints follow REST conventions.
Vite	A modern frontend build tool that provides fast development server startup and hot module replacement. Used in InterviewACE as an alternative to Create React App.

Table I.1: Glossary of Terms Used in This Report

COURSERA CERTIFICATION

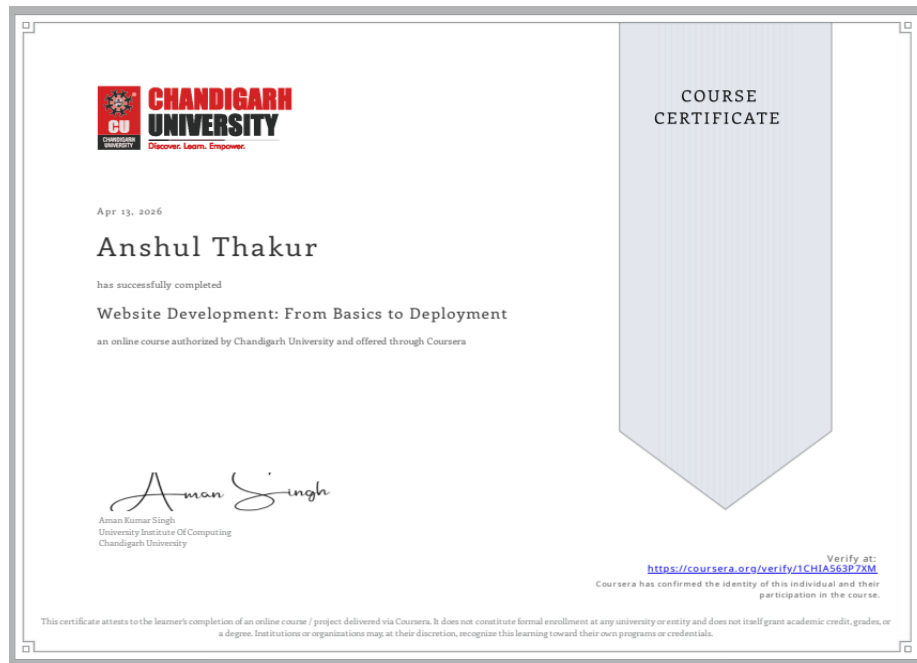


Figure 14: Coursera Certification 1



Figure 15: Coursera Certification 2

GitHub Repository: <https://github.com/pankajkatoch07/InterviewAce.git>